

TEMA 6

MÁQUINAS DE ESTADO FINITAS (FSM) Y SU DESCRIPCIÓN EN VHDL



ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA
Universidad de Córdoba



DEPARTAMENTO DE
INGENIERÍA ELECTRÓNICA
Y DE COMPUTADORES
UNIVERSIDAD DE CÓRDOBA



Tema 1. Análisis de los circuitos integrados digitales



Tema 2. Diseño de sistemas digitales mediante Dispositivos Lógicos Programables



Tema 3. Introducción al lenguaje VHDL



Tema 4. Statements concurrentes, tipos de datos numéricos y conversiones en el lenguaje VHDL



Tema 5. Procesos y atributos en VHDL



Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL



Tema 7. Diseño jerárquico y parametrizable en VHDL



Tema 8. Diseño RTL en VHDL



Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL

❖ Objetivos

- Analizar la estructura general de las Máquinas de Estados Finitas (FSM).
- Comprender el funcionamiento de las FSMs.
- Asimilar los tipos de FSMs y sus características, tanto de funcionamiento como de diseño.
- Indicar las fases de la metodología clásica de diseño de las FSM.
- Reseñar la importancia de la síntesis óptima del diagrama de estados.
- Adquirir habilidad en la síntesis de los diagramas de estados.
- Comprender el formato de descripción en VHDL de las FSM, tanto del tipo Mealy como Moore.

Tema 6. Máquinas de estados finitas (FSM) y su descripción en VHDL

❖ Bibliografía

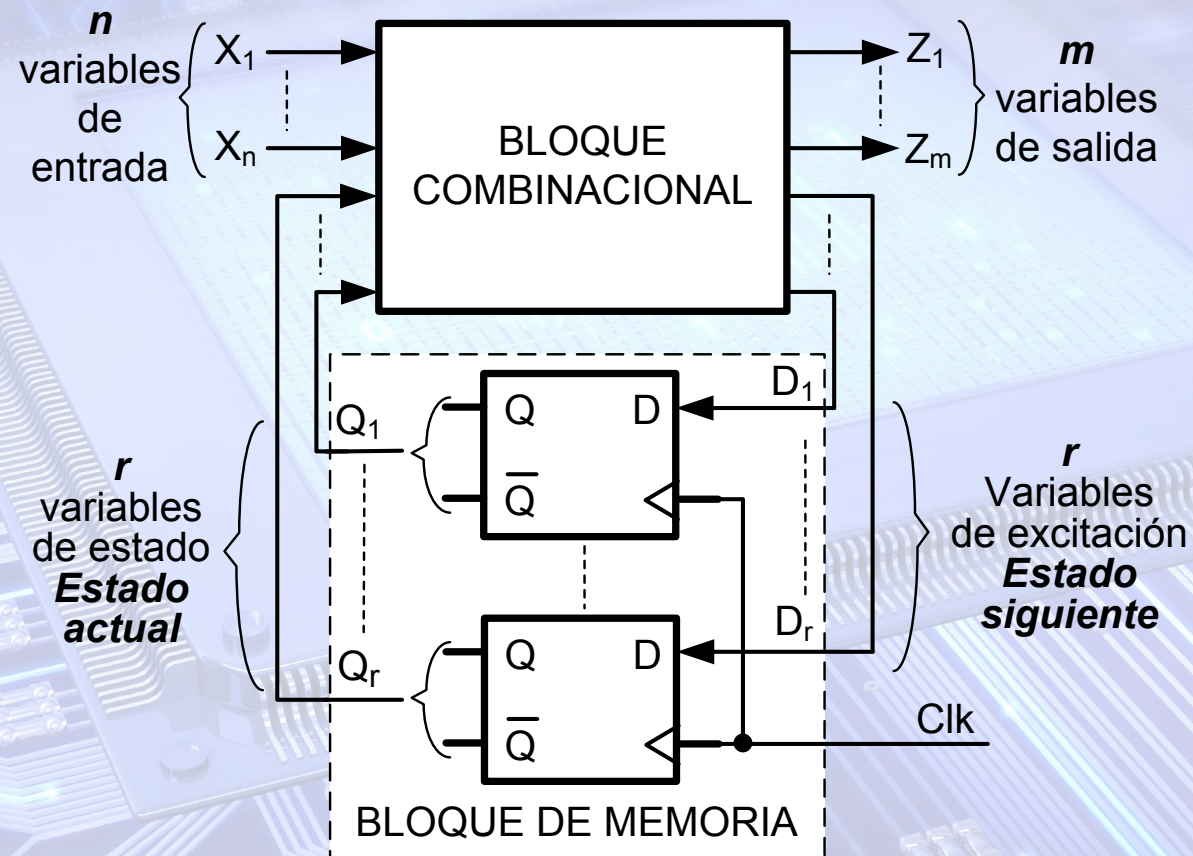
- John F. Wakerly. Diseño Digital. Principios y Prácticas. Ed. Prentice Hall. 2001.
- Lloris-Prieto. Diseño Lógico. Ed. Mc Graw-Hill. 1996.
- Gajski. Principios de Diseño digital. Ed. Prentice Hall. 1997.
- Charles H. Roth. Fundamentos de Diseño Lógico.
- William Kafig. VHDL 101. Everything you need to know to get started. Ed. Newnes. 2011.
- Kevin Skahill. VHDL for Programmable logic. Ed. Addison-Wesley. 1996.
- Pong P. Chu. RTL Hardware Design Using VHDL. Ed. Wiley. 2006.
- Pong P. Chu. FPGA Prototyping by VHDL Examples. Ed. Wiley. 2008.
- Ion Grout. Digital Systems Design with FPGAs and CPLDs. Ed Elsevier Ltd. 2008.
- David Pellerin y otros. VHDL Made Easy!. Ed. Mc Graw Hill. 1997.
- Douglas Perry. VHDL. Ed. Mc-Graw Hill. 1998.
- R. C. Cofer and Benjamin F. Harding. Rapid System Prototyping with FPGAs. Ed. Elsevier Inc. 2006.

1. Introducción

- ❖ En algunos sistemas digitales su funcionamiento consiste en **realizar una secuencia de pasos**:
 - **Detección de una secuencia o patrón de bits** que se recibe en serie por una o varias entradas.
 - ✓ Ejemplos:
 - Detector de tres bits con valor 1 (no tienen que ser seguidos).
 - Detector de un número BCD concreto (Ejemplo: 0101 = 5).
 - **Generar una secuencia específica de valores de salida**.
 - ✓ Ejemplos:
 - Generar la secuencia de salida 10011 ... (una única salida).
 - Generar la secuencia de salida 3, 5, 7, 2 ... (tres salidas).
 - El diseño de estos sistemas se realiza mediante una **Máquina de Estados Finita (FSM)**.

2. Estructura y funcionamiento de las FSM

- En los diseños realizados con PLDs se usan FSM síncronas.
- Estructura general de las Máquinas de Estados Finitas Síncronas.

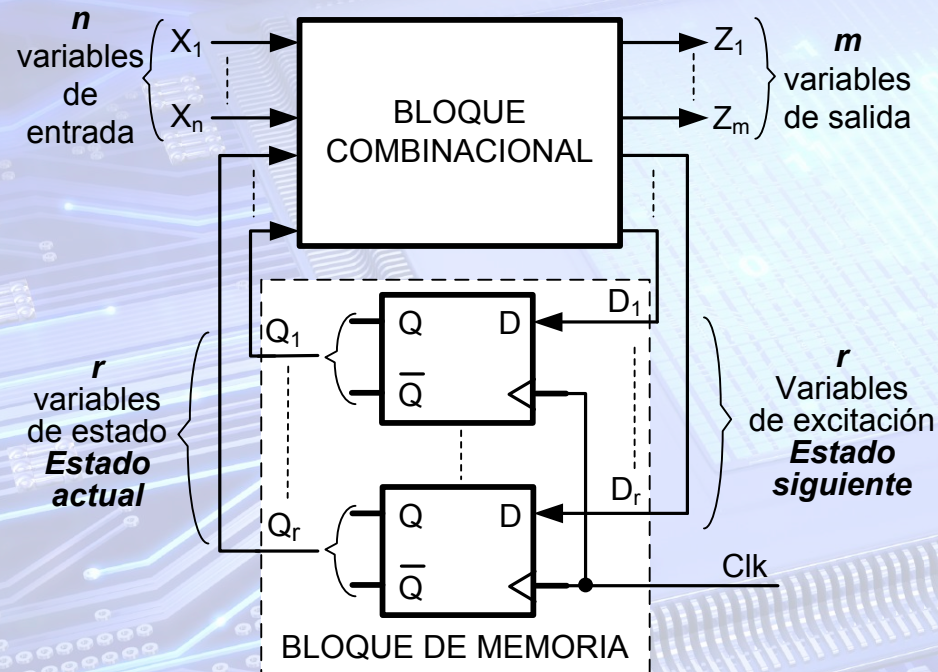


2. Estructura y funcionamiento de las FSM

❖ Estructura general de las FSM síncronas

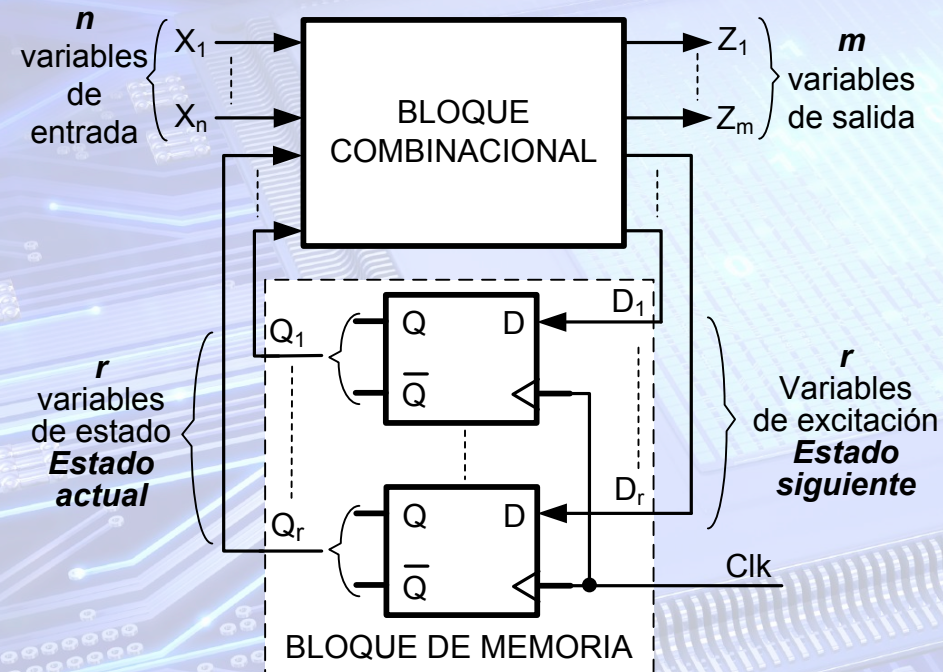
■ Bloque de Memoria:

- ✓ La memoria almacena información sobre la evolución del sistema.
- ✓ Esta información se representa mediante r señales binarias ($Q_r \dots Q_1$), denominadas **variables de estado**.
 - Corresponden a las salidas del bloque de memoria.
- ✓ Cada una de las 2^r combinaciones binarias de las r variables de estado se denomina **estado del sistema**.
 - Un estado almacena información sobre la evolución del sistema.
- ✓ En las FSM síncronas el bloque de memoria consta de flip-flops (FF) activos por flanco, que usan la misma señal de reloj (Clk).
- ✓ En las FSM el bloque de memoria se suele denominar registro de estado.



2. Estructura y funcionamiento de las FSM

❖ Estructura general de las FSM síncronas



■ Bloque Combinacional

- A partir de las **n variables de entrada** y las **r variables de estado** determina los valores de las **m variables de salida** y las **p variables de excitación**.
- Las **variables de excitación** especifican el estado al que evolucionará el sistema, y que se almacenará en la memoria con el flanco activo de la señal de reloj, flanco de subida en el ejemplo de la figura.
- El estado en el que se encuentra el sistema en un instante de tiempo dado, se denomina **estado actual**, y al que evolucionará **estado siguiente**.

2. Estructura y funcionamiento de las FSM

Diagrama de bloques de una FSM con restauración asíncrona

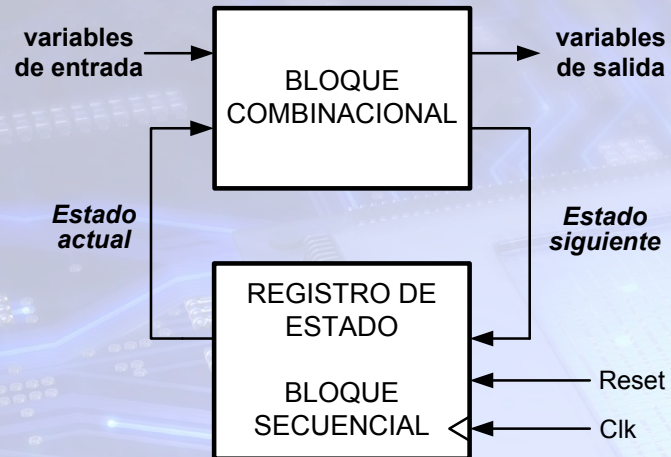
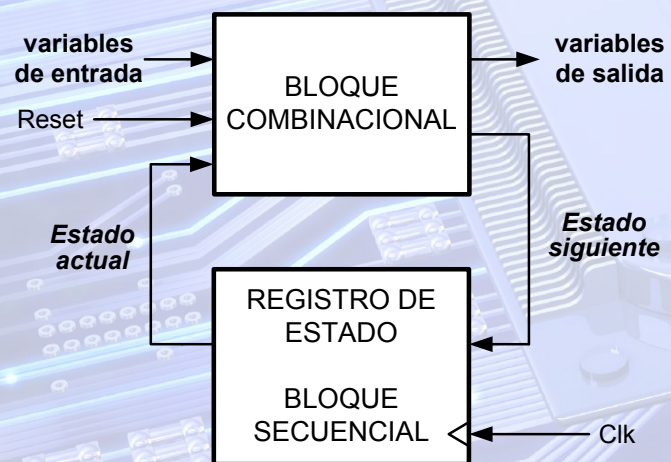


Diagrama de bloques de una FSM con restauración síncrona



❖ Funcionamiento

- Una FSM trabaja en **dos fases**:
 - ✓ La FSM está en un **estado actual** dado de su secuencia:
 - El **bloque combinacional** determina, en función del valor de las entradas en ese estado actual, el **valor de las salidas y el estado siguiente**.
 - ✓ Al producirse un flanco activo se **almacena el estado siguiente** en el registro de estado.
 - Pasa a ser su nuevo **estado actual**.
- **Señal de restauración o inicialización (Reset)**
 - ✓ Al activarse fuerza a la FSM a ir a su estado inicial.
 - ✓ **Dos tipos**:
 - **Asíncrona**. Figura superior:
 - Es independiente de la señal de reloj.
 - Se conecta a las entradas de clear y/o preset de los Flip-flops.
 - **Síncrona**. Figura inferior:
 - Depende de la señal de reloj.
 - Forma parte de las funciones de excitación que determinan el estado siguiente.

2. Estructura y funcionamiento de las FSM

❖ Tipos de Autómatas Finitos

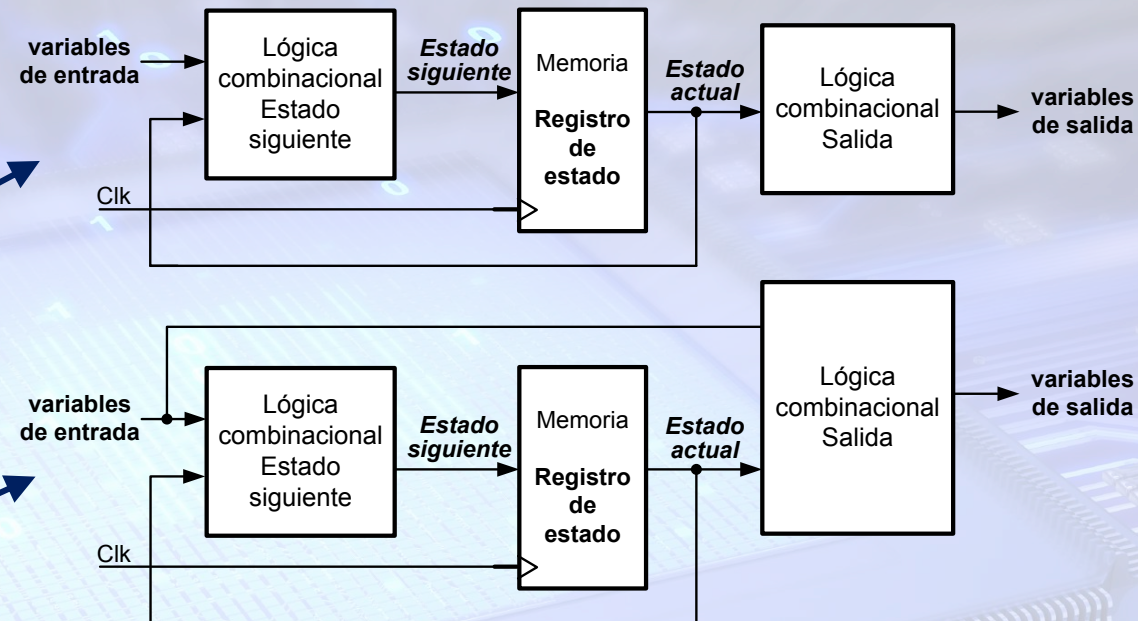
- Según las variables que determinen el valor de las variables de salida:

✓ Autómata Moore

- El valor de las salidas depende sólo del **estado actual** en el que se encuentre el sistema.

✓ Autómata Mealy

- El valor de las salidas depende tanto del **estado actual** en el que se encuentre el sistema como del **valor que tienen las señales de entrada** en ese estado actual.



- El **autómata Mealy** puede cambiar de valor sus salidas un ciclo antes que el **Moore**, ya que:

- ✓ La máquina Mealy responde inmediatamente al cambiar las entradas.
- ✓ La máquina Moore tiene que cambiar primero de estado, por lo que debe esperar un ciclo de reloj antes de que cambien de valor sus salidas.

3. Tipos de Máquinas de Estados Finitas (FSM). Diagrama de estados de los Autómatas Mealy y Moore

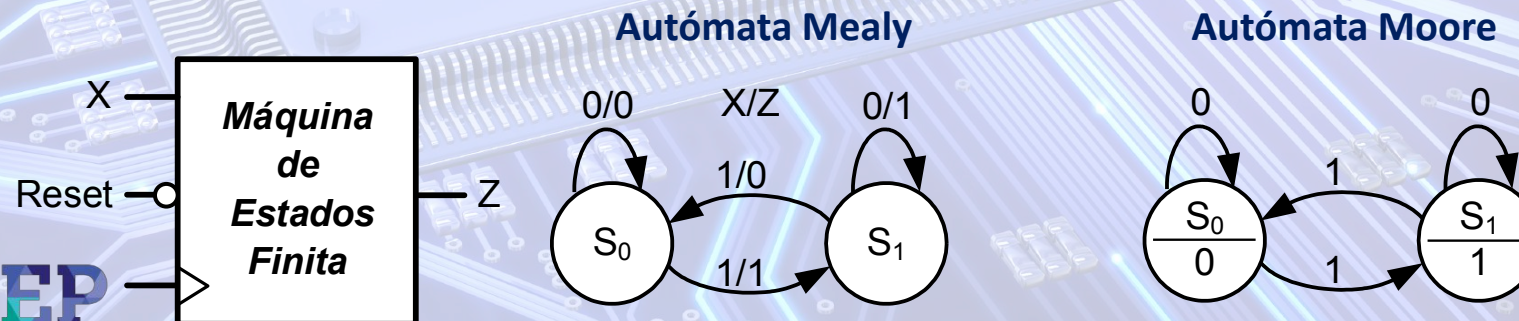
- ❖ Los autómatas Mealy y Moore solamente se diferencian en la forma de generar los valores de las señales de salida.
- ❖ Por tanto, el formato de sus diagramas de estados sólo se diferenciará en la representación del valor de las salidas.

■ Autómata Mealy

- El valor de las señales de salida depende del estado actual y del valor que tienen las entradas en dicho estado actual.
- Se indica el valor de las salidas a continuación del de las entradas junto a las flechas que describen las transiciones de estados, según el formato: **Valor_entradas/Valor_salidas**.

■ Autómata Moore

- El valor de las señales de salida depende sólo del estado actual.
- Se indica el valor de las salidas dentro del círculo de cada uno de los estados.



3. Tipos de Máquinas de Estados Finitas (FSM). Diagrama de estados de los Autómatas Mealy y Moore

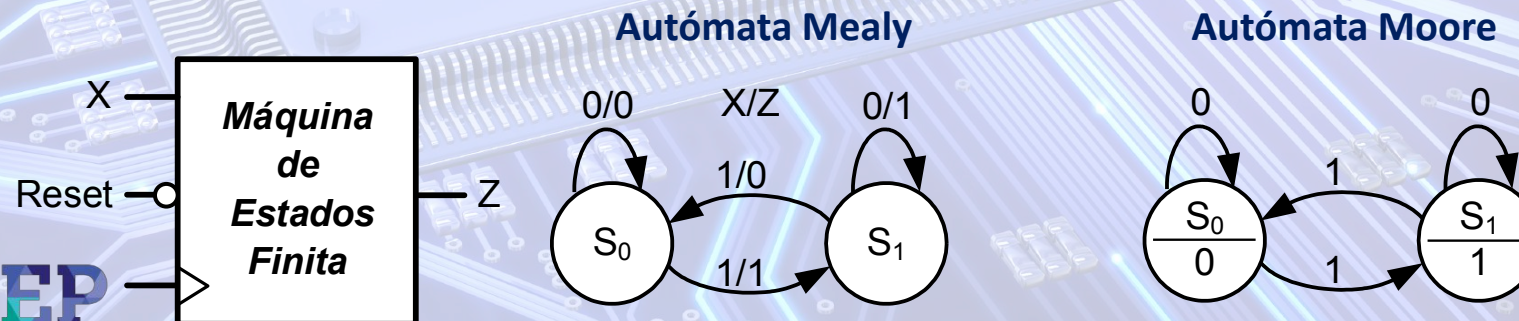
- ❖ Los autómatas Mealy y Moore solamente se diferencian en la forma de generar los valores de las señales de salida.
- ❖ Por tanto, el formato de sus diagramas de estados sólo se diferenciará en la representación del valor de las salidas.

■ Autómata Mealy

- El valor de las señales de salida depende del estado actual y del valor que tienen las entradas en dicho estado actual.
- Se indica el valor de las salidas a continuación del de las entradas junto a las flechas que describen las transiciones de estados, según el formato: **Valor_entradas/Valor_salidas**.

■ Autómata Moore

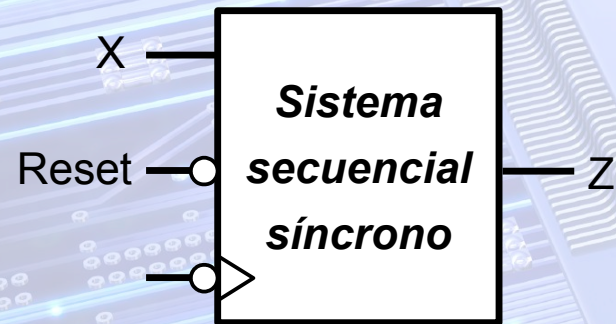
- El valor de las señales de salida depende sólo del estado actual.
- Se indica el valor de las salidas dentro del círculo de cada uno de los estados.



4. Metodología general de síntesis de FSM.

Descripción funcional.

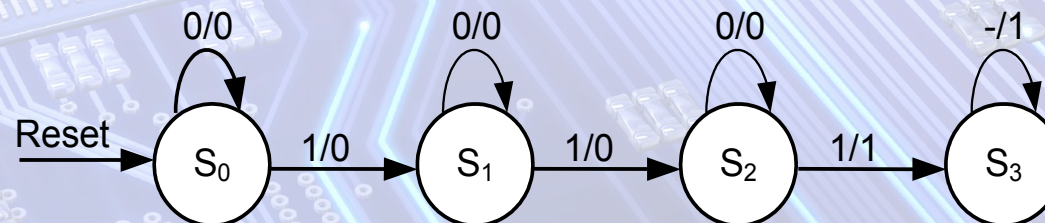
- ❖ Se describirá la metodología de diseño mediante un ejemplo sencillo.
- ❖ Descripción funcional.
 - La primera fase del diseño es la especificación funcional del circuito.
 - Es una **descripción en lenguaje natural de las funciones** que debe realizar el circuito a partir de sus señales de entrada.
 - **Ejemplo.**
 - ✓ Diseñar el circuito secuencial síncrono de la figura. Por la **entrada X** se reciben bits en **serie** sincronizados con el flanco de bajada de la señal de reloj, **Clk**. **Reset** es una entrada de **restauración asíncrona** activa a nivel bajo. La **salida Z** se pondrá a 1 al recibir 3 bits con valor 1, teniendo en cuenta que éstos no tienen por qué ser seguidos. Activará la salida **al recibir el tercer bit** con valor 1 y **quedará bloqueado** con la **salida activa** hasta que se **restaure** con la señal asíncrona **Reset**.



4. Metodología general de síntesis de FSM. Diagrama de estados.

❖ Diagrama de estados del diseño propuesto.

- Se diseña como un **autómata Mealy**.
- La señal de restauración **Reset** es **asíncrona**.
 - ✓ Como después se verá se conecta a los clear o a los preset de los flip-flops dependiendo de la asignación que se realice al estado inicial (que se denota como S_0).
 - ✓ Por tanto, no se tiene en cuenta en la obtención de las funciones de excitación.
 - ✓ No obstante, es conveniente indicarlo en el diagrama de estados:
 - Se pone una flecha entrante al estado inicial con el nombre de la señal de restauración.
- El formato será **X/Z**.
 - ✓ Cada vez que se define una transición, hay que determinar si se puede pasar a un estado ya definido, o por el contrario, hay que definir otro nuevo.
 - ✓ En cada estado deben de quedar definidas todas las posibles transiciones válidas: como máximo 2^n , siendo n el número de señales de entrada.



4. Metodología general de síntesis de FSM.

Tabla de transición

❖ Tabla de estados

S ^v	X ^v	
	0	1
S ₀	S _{0,0}	S _{1,0}
S ₁	S _{1,0}	S _{2,0}
S ₂	S _{2,0}	S _{3,1}
S ₃	S _{3,1}	S _{3,1}

S^{v+1}, Z^v

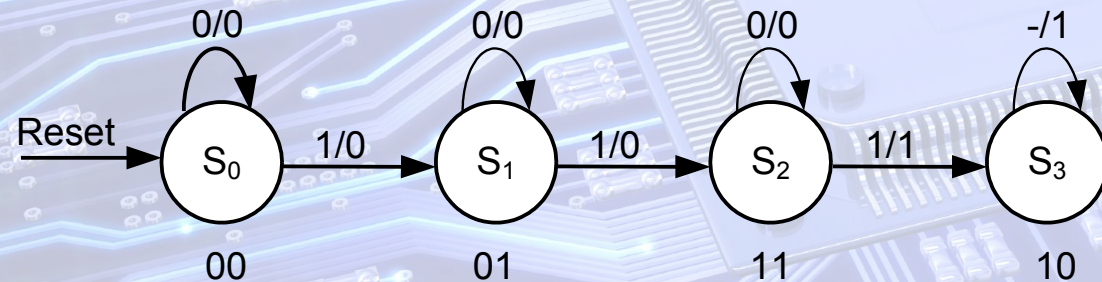
❖ Tabla de transición.

- Se determina el número de **variables de estado** (r):
 - Se selecciona el menor valor de r que cumple la inecuación:
 - $M \leq 2^r$, siendo M el nº de estados.
 - En el ejemplo: $4 \leq 2^r \Rightarrow r = 2$
 - Las variables serán Q₁ y Q₀.
- Se asigna a cada estado una de las 2^r (2² = 4) combinaciones binarias de las variables de estado.
 - Al estado inicial se le suele asignar la combinación 0.
 - Al resto de estados hay varias opciones. Se podrá usar cualquiera de las dos siguientes:
 - El equivalente binario del subíndice numérico del estado, o
 - Las combinaciones binarias de forma que se minimice el número de transiciones de las variables de estado:
 - ✓ Que cambie una única variable de estado al pasar de un estado a otro.
 - ✓ No siempre se puede conseguir.

4. Metodología general de síntesis de FSM.

Tabla de transición

S^v	$Q_1^v \ Q_0^v$	X^v			
		0	1	0	1
S_0	0 0	0 0	0 1	0	0
S_1	0 1	0 1	1 1	0	0
S_2	1 1	1 1	1 0	0	1
S_3	1 0	1 0	1 0	1	1

 $Q_1^{v+1} \ Q_0^{v+1} \ Z^v$


❖ Tabla de transición del diseño propuesto

- Se asignarán los estados de forma que se minimice el número de transiciones de las variables de estado.
 - ✓ Al ser la secuencia de estados fija se puede hacer la asignación de estados de forma que cambie una única variable de estado al pasar de un estado a otro.
 - Realmente se hace una asignación en **código Gray**.
 - ✓ Si de un estado se pasa a varios estados diferentes no siempre se puede conseguir realizar esta asignación tan idónea.
 - No obstante, siempre se podrá realizar de forma que se minimice el número de transiciones de las variables de estado.

4. Metodología general de síntesis de FSM. Tabla de excitación

❖ Tabla de excitación.

- Se selecciona primero el tipo de flip-flop que se va a usar.
 - ✓ Se emplearán flip-flops JK.
- Las variables de excitación serán:
 - ✓ J_0 y K_0 , que corresponden a las entradas del FF que genera la variable de estado Q_0 .
 - ✓ J_1 y K_1 , que corresponden a las entradas del FF que genera la variable de estado Q_1 .
- Para obtener las tablas de excitación del sistema hay que emplear la tabla de excitación del FF JK.
- A continuación, se indican las tablas de excitación de los flip-flops JK, T, D y SR.

$Q^V \rightarrow Q^{V+1}$	J	K	$Q^V \rightarrow Q^{V+1}$	T	$Q^V \rightarrow Q^{V+1}$	D	$Q^V \rightarrow Q^{V+1}$	S	R
0 → 0	0	-	0 → 0	0	0 → 0	0	0 → 0	0	-
0 → 1	1	-	0 → 1	1	0 → 1	1	0 → 1	1	0
1 → 0	-	1	1 → 0	1	1 → 0	0	1 → 0	0	1
1 → 1	-	0	1 → 1	0	1 → 1	1	1 → 1	-	0

4. Metodología general de síntesis de FSM. Tabla de excitación

■ Tabla de excitación

✓ Se puede formar junto a la tabla de transición o independientemente.

■ Funciones de excitación y de salida.

✓ Dado la asignación de estados, las tablas de excitación de cada variable y la de la salida Z tiene el mismo formato que un mapa de Karnaugh de 3 variables.

✓ Se puede simplificar las funciones de excitación y de salida directamente en la tabla.

$Q_1^v Q_0^v$		X^v											
		0	1	0	1	0	1	0	1	0	1	0	1
0	0	00	01	0	0	0	1	-	-	0	0	-	-
0	1	00	01	0	0	-	-	0	0	0	1	-	-
1	1	11	10	0	1	-	-	0	1	-	-	0	0
1	0	10	10	1	1	0	0	-	-	-	-	0	0
		$Q_1^{v+1} Q_0^{v+1}$		Z^v		J_0		K_0		J_1		K_1	

$Q^v \rightarrow Q^{v+1}$	J	K
0 $\rightarrow$ 0	0	-
0 $\rightarrow$ 1	1	-
1 $\rightarrow$ 0	-	1
1 $\rightarrow$ 1	-	0

$$J_0 = \bar{Q}_1 X \quad K_0 = Q_1 X$$

$$J_1 = Q_0 X \quad K_0 = 0$$

$$Z = Q_1 X + Q_1 \bar{Q}_0$$

Obtención de la tabla de excitación

■ Las variables de excitación J_0 y K_0 están asociadas a las variables de estado Q_0 .

■ De igual manera, J_1 y K_1 con Q_1 .

■ Si el sistema está en el estado 01, se mantendrá en el mismo estado si $X = 0$.

$$Q_0^v(1) \rightarrow Q_0^{v+1}(1) \Rightarrow J_0 = - \text{ y } K_0 = 0$$

$$Q_1^v(0) \rightarrow Q_1^{v+1}(0) \Rightarrow J_1 = 0 \text{ y } K_1 = -$$

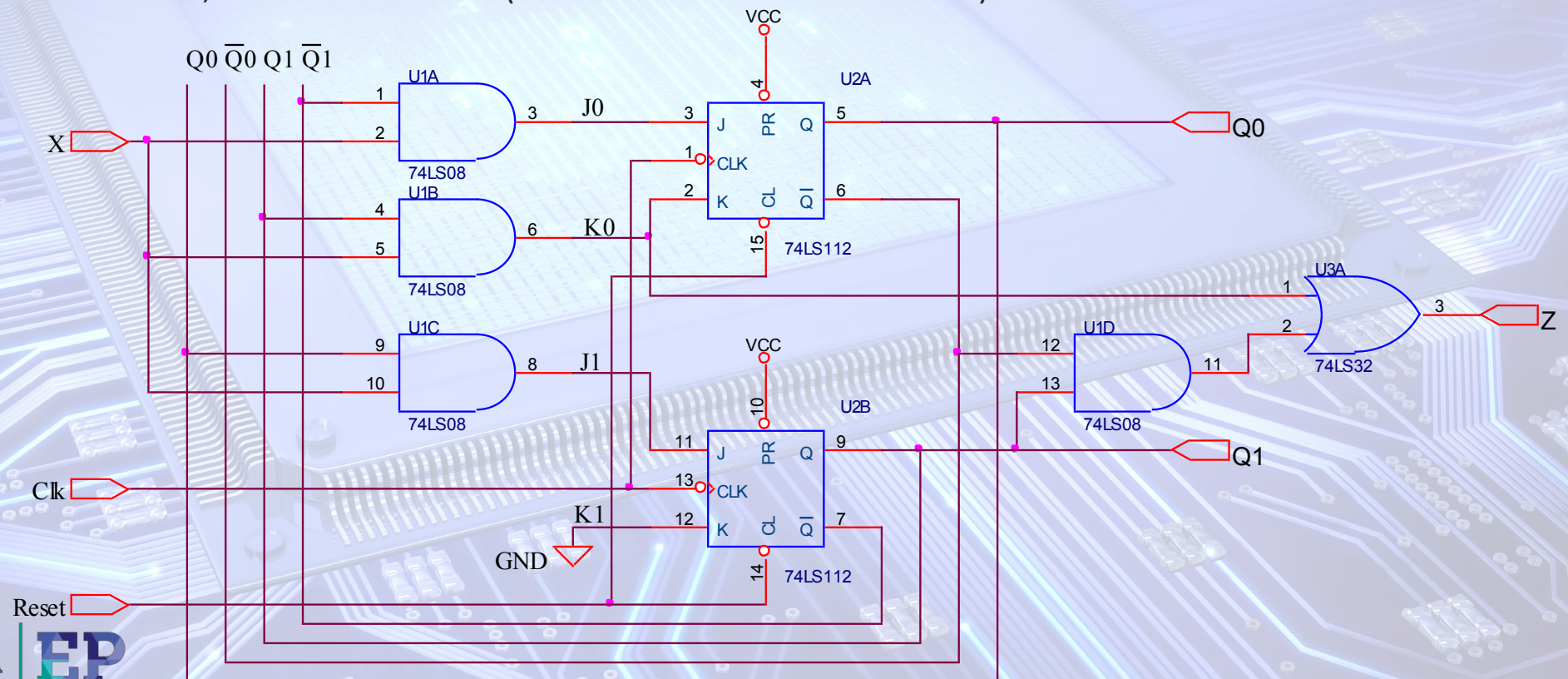
■ Si el sistema está en el estado 01, el estado siguiente es 11 si $X = 1$.

$$Q_0^v(1) \rightarrow Q_0^{v+1}(1) \Rightarrow J_0 = - \text{ y } K_0 = 0$$

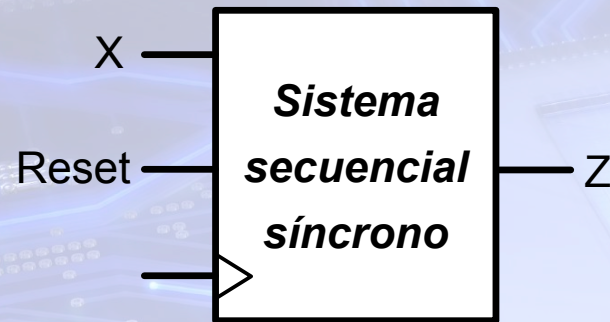
$$Q_1^v(0) \rightarrow Q_1^{v+1}(1) \Rightarrow J_1 = 1 \text{ y } K_1 = -$$

4. Metodología general de síntesis de FSM. Diagrama lógico

- Se implementan mediante puertas lógicas las funciones de excitación y de salida obtenidas.
- Como reset es asíncrona y el estado inicial es 00, se conecta a las entradas de clear de ambos flip-flops.
- Al no usarse las entradas de preset de los flip-flops se tienen que conectar a su nivel desactivo, es decir a nivel alto (VCC = + tensión de alimentación).



5. Ejemplo de diseño. Diferencias entre los autómatas Mealy y Moore



- ¿Qué quiere decir que se generará un pulso a nivel alto en la salida?
 - ✓ Z se pondrá a nivel alto durante un único periodo de reloj.

❖ Diseñar un circuito secuencial síncrono con dos entradas, Reset y X, y una salida Z. Reset es la entrada de restauración síncrona y es activa a nivel lógico alto. Por X se recibe continuamente bits en serie sincronizados con el flanco de subida de la señal de reloj. La salida Z generará un pulso a nivel alto cada vez que en la entrada X se detecte la secuencia 0011.

- Se trata de detector de secuencia que puede ir precedida de cualquier patrón de bits
 - ✓ Se debe conocer el orden de recepción de los bits.
 - Supondremos que primero se recibe el bit de la izquierda (primer 0 de la secuencia)
 - ✓ La salida Z se activa si los últimos cuatro bits recibidos son 0011 (los de la secuencia).
 - ✓ Ejemplos:

Secuencia 1ª

X	0	0	1	0	0	1	1	0	1	1	0	0	1	1
Z	0	0	0	0	0	0	1	0	0	0	0	0	0	1

Secuencia 2ª

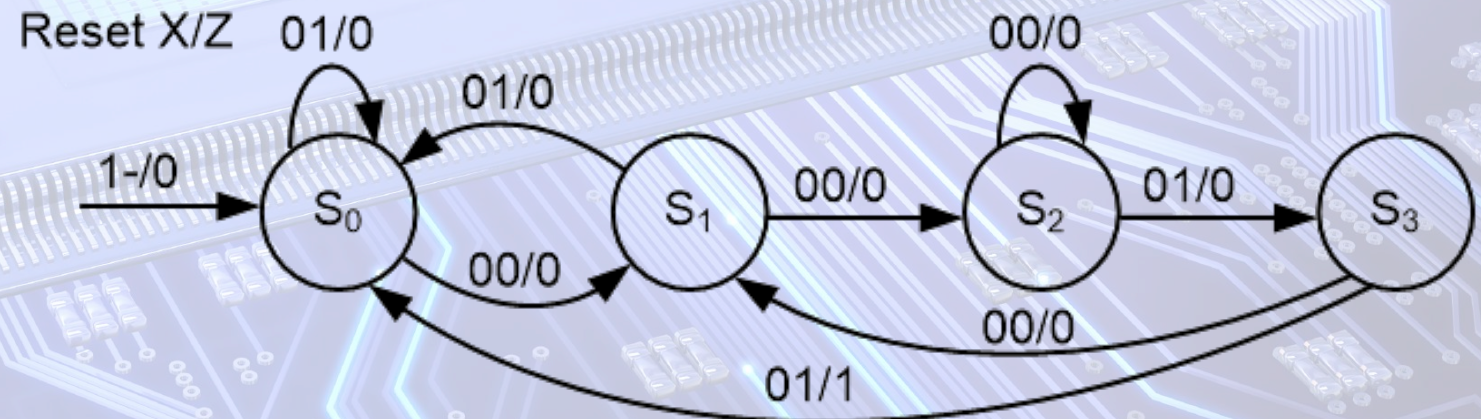
5. Ejemplo de diseño. Diferencias entre los autómatas Mealy y Moore

Representación abreviada de la señal de restauración o inicialización síncrona.

- Si se activa Reset (nivel alto) el sistema debe ir al estado inicial indiferentemente de su estado actual y del valor de X.
- Por tanto, esta condición se indica mediante una flecha entrante al estado inicial, y junto a ésta el valor de las entradas, Reset y X (1-).
- Para el resto de transiciones se debe de indicar, que Reset tiene que estar desactiva y el valor de X para cada transición (0X).

❖ Diagrama de estados Mealy (Secuencia 0011)

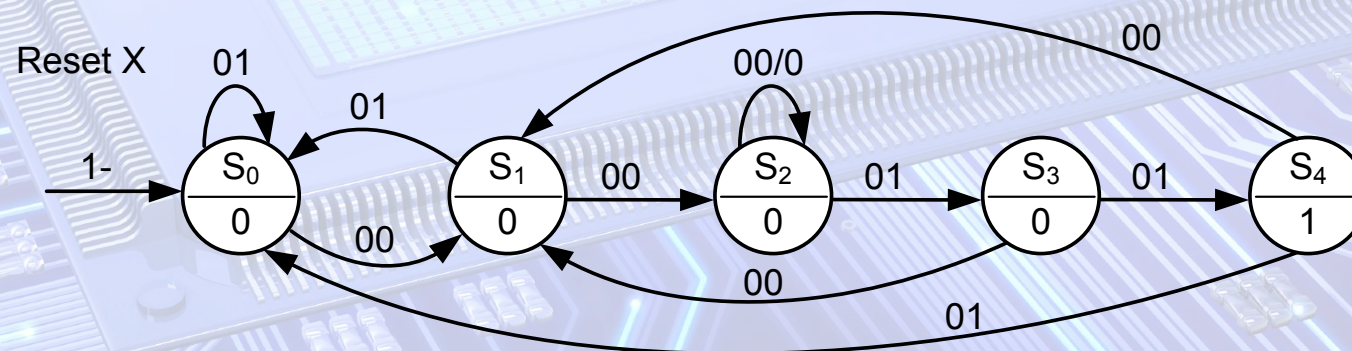
- Cada estado recuerda los bits válidos de la secuencia que se llevan recibidos.
- Si se rompe la secuencia se debe analizar los últimos bits recibidos para ver cuales sirven como inicio de la secuencia.
- Si se puede aprovechar algunos bits se va al estado que recuerda esa secuencia de bits recibida
- Por ser síncrona la señal de restauración Reset, ésta debe formar parte de las funciones de excitación de los flip-flops:
 - Debe incluirse en todo el proceso de diseño.



5. Ejemplo de diseño. Diferencias entre los autómatas Mealy y Moore

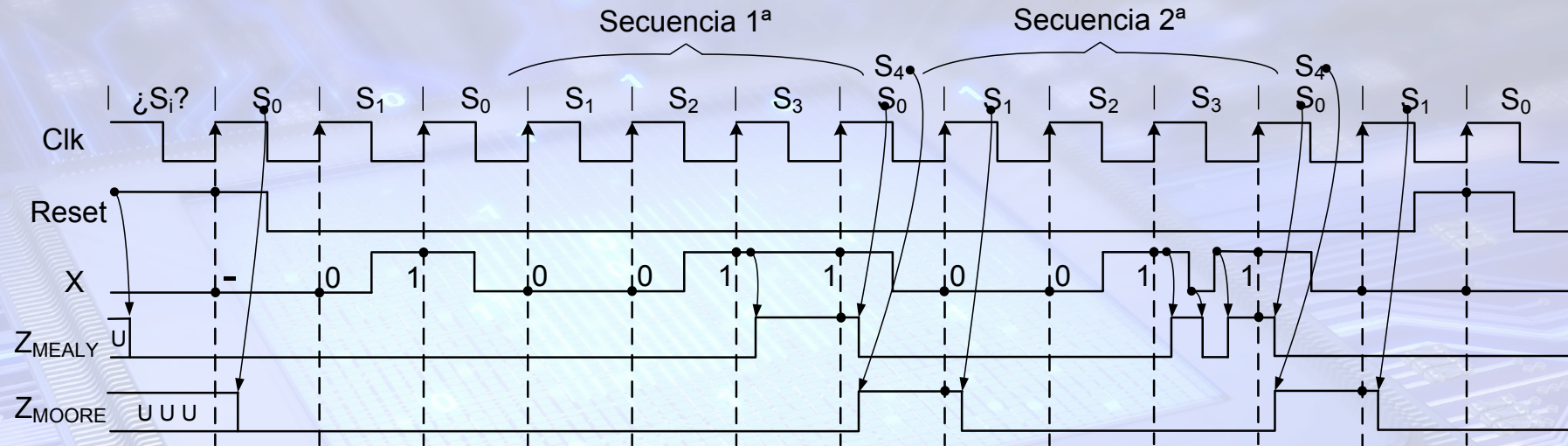
- **Diagrama de estados Moore**

- Cada estado recuerda los bits válidos de la secuencia que se llevan recibidos.
- Como la salida Z depende del estado actual se activará cuando el autómata esté en un estado que recuerda que se ha recibido la secuencia (S_4).
- La salida Z se activa después de recibirse la secuencia 0011.
- Una vez recibida la secuencia se pasa al estado inicial si se recibe un 1, o a S_1 si es un 0, ya que sería el primer bit de otra secuencia.



5. Ejemplo de diseño. Diferencias entre los autómatas Mealy y Moore

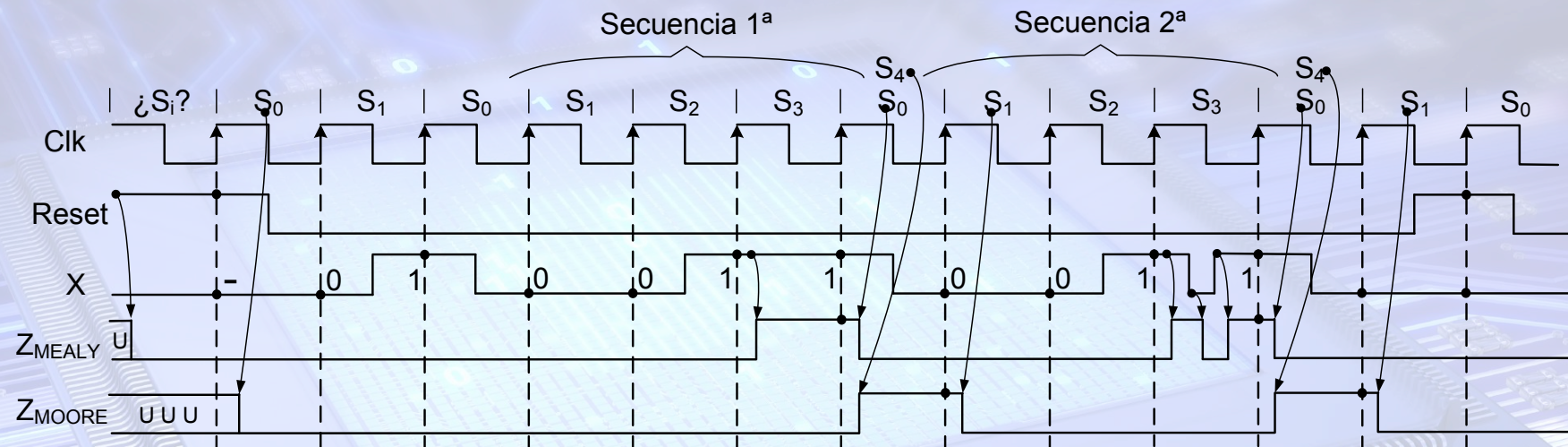
❖ Cronogramas del funcionamiento de ambos autómatas



- Por ser Reset síncrona ambos autómatas no se inicializan al estado S_0 hasta el primer flanco de subida en el que Reset está activa.
- En el autómata Mealy la salida Z depende de las entradas, Reset y X, y del estado actual:
 - ✓ La salida Z se desactiva si Reset se activa.
 - ✓ Se activa en S_3 si $X = 1$, pero si X cambia a 0 se desactiva.
 - Se pueden generar pulsos espureos en la salida
 - ✓ Como regla general en un autómata Mealy si se activa la señal de restauración o inicialización, se desactivan todas sus salidas.

5. Ejemplo de diseño. Diferencias entre los autómatas Mealy y Moore

❖ Cronogramas del funcionamiento de ambos autómatas



- En el autómata Moore la salida Z depende sólo del estado actual:
 - ✓ La salida Z se desactiva al ir al estado S₀ si Reset se activa.
 - ✓ Se activa en S₄ después de recibirse la secuencia 0011.
 - ✓ La salida Z está sincronizada con la señal de reloj.
 - No se generán pulsos espureos en la salida.
 - ✓ De S₄ se pasa a S₁ si se recibe un 0.
 - Es primer bit válido de otra secuencia 0011

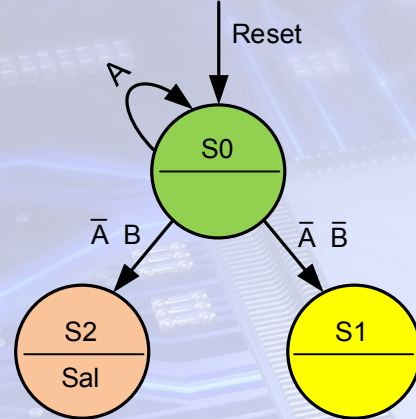
5. Ejemplo de diseño. Diferencias entre los autómatas Mealy y Moore

❖ Conclusión

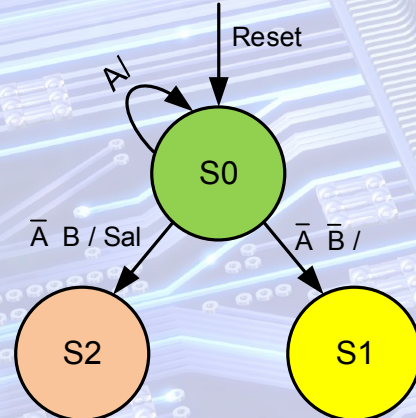
- ✓ Por tanto, si el diseño especifica que la salida se tiene que activar al recibir el último bit de la secuencia, habrá que diseñarlo por Mealy, y si es después de recibir el último bit de la secuencia se podrá realizar por Mealy o por Moore.

6. Descripción de FSM. Formatos de representación

Ejemplo de Diagrama de Estados abreviado FSM Moore



Ejemplo de Diagrama de Estados abreviado FSM Mealy



❖ Formatos de representación del funcionamiento de una FSM:

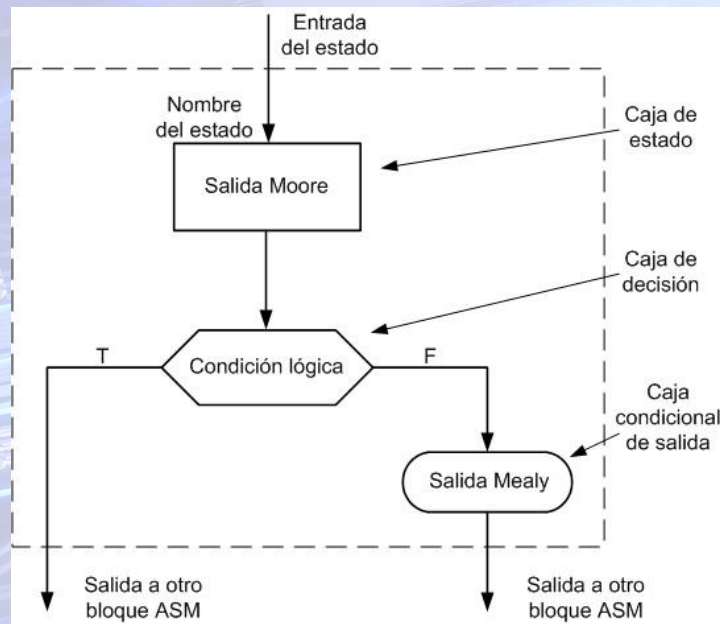
- Diagrama de Estados (DE).
- Diagrama de Máquina de Estados Algorítmica (ASM) o Diagrama de Flujo.

❖ Formatos abreviados:

- Para facilitar la representación de los DE y los ASM, y para obtener más fácilmente su descripción VHDL, se usará un formato abreviado.
- La condición de **reset** se indicará mediante una **flecha entrante al estado inicial**, tanto para el tipo asíncrono como síncrono.
- Las **condiciones de transición de estados** se indica mediante una **expresión lógica de las entradas**.
 - ✓ Se **complementa** la entrada que debe estar **desactiva**.
 - ✓ **No se complementa** la entrada que debe estar **activa**.
- Los valores de las salidas:
 - ✓ **Por defecto, todas las salidas están desactivas.**
 - ✓ **Sólo se indicará el nombre de la señal que se active.**
 - Si es una **FSM Moore**, se pone el nombre de las señales en los estados correspondientes.
 - Si es una **FSM Mealy**, se indica el nombre de las señales a continuación de la / en las flechas de transición.

6. Descripción de FSM. Formatos de representación. Diagramas ASM

Modelo general de un
bloque ASM



❖ Características generales del Diagrama ASM.

- ✓ Representa la misma información que el Diagrama de Estados, pero de una forma más descriptiva.
- ✓ Permite describir de una forma más clara la secuencia de pasos de un algoritmo y una secuencia de microoperaciones.
- ✓ Se usará en la descripción de las **FSMD** (FSM con una ruta de datos) en el tema 8.

❖ Formato del Diagrama ASM.

- ✓ Consta de una red de bloques ASM.
- ✓ Un bloque ASM consta de una caja de estado y una red opcional de cajas de decisión y de salidas condicionales.
- ✓ La caja de estado representa un estado de la FSM.
 - Se identifica por un nombre simbólico en la esquina superior izquierda de la caja de estado.
 - Para una **FSM Moore** el nombre de la acción o salida indicada dentro de la caja, especifica la acción o la salida que se activa en ese estado.

6. Descripción de FSM. Formatos de representación. Diagramas ASM

❖ Formato del Diagrama ASM.

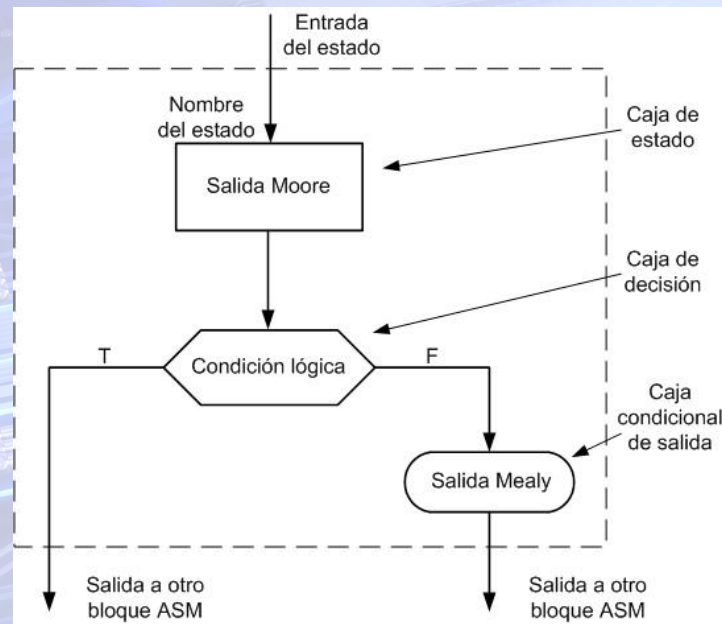
- Una caja de decisión comprueba una condición de entrada para determinar el camino de salida del actual bloque ASM.

- ✓ Contiene una expresión lógica de las señales de entrada.
- ✓ Realiza la misma función que la expresión lógica de las flechas de los diagramas de estados.

✓ Funcionamiento.:

- Dependiendo del valor de la expresión lógica puede seguir por el camino verdad o por el camino falso, que se etiquetan T o F en los caminos de salida.
- Se pueden conectar entre sí múltiples cajas de decisión para describir condiciones complejas, como $(a > b)$ and $(c \neq 1)$.

Modelo general de un
bloque ASM



6. Descripción de FSM. Formatos de representación. Diagramas ASM

❖ Formato del Diagrama ASM.

- Una caja condicional de salida indica las señales de salida que se activan.

✓ Sólo se puede poner después de un camino de salida de una caja de decisión.

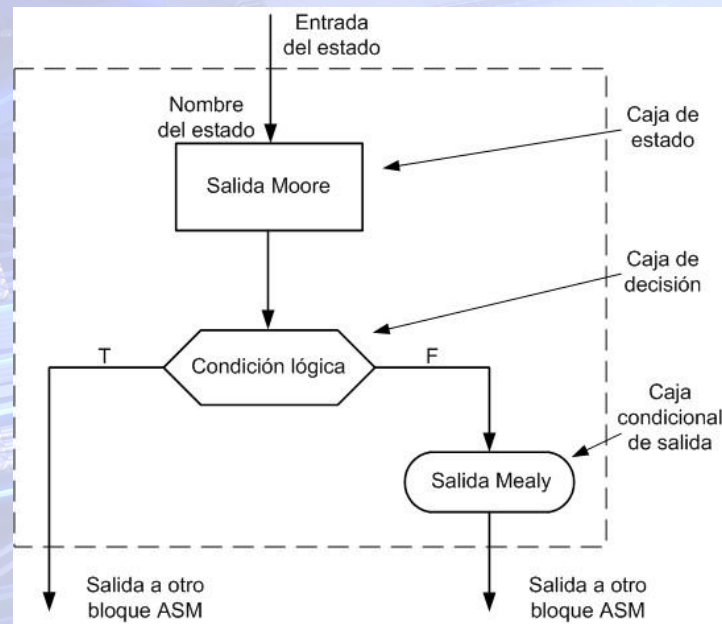
✓ Funcionamiento.

- Las señales de salida sólo se activan si se cumple o no la condición de salida de la caja de decisión previa, según al camino de salida que se conecte.

✓ Características.

- Como la condición es una expresión lógica de las entradas, las señales de salida dependen del estado actual y del valor de las señales de entrada.
- Por tanto, **se usan para describir las FSM Mealy.**

Modelo general de un
bloque ASM



6. Descripción de FSM. Formatos de representación. Diagramas ASM

❖ Ejemplo de conversión de DE a ASM

- Una FSM puede generar al mismo tiempo salidas Mealy y Moore.

Diagrama de estados

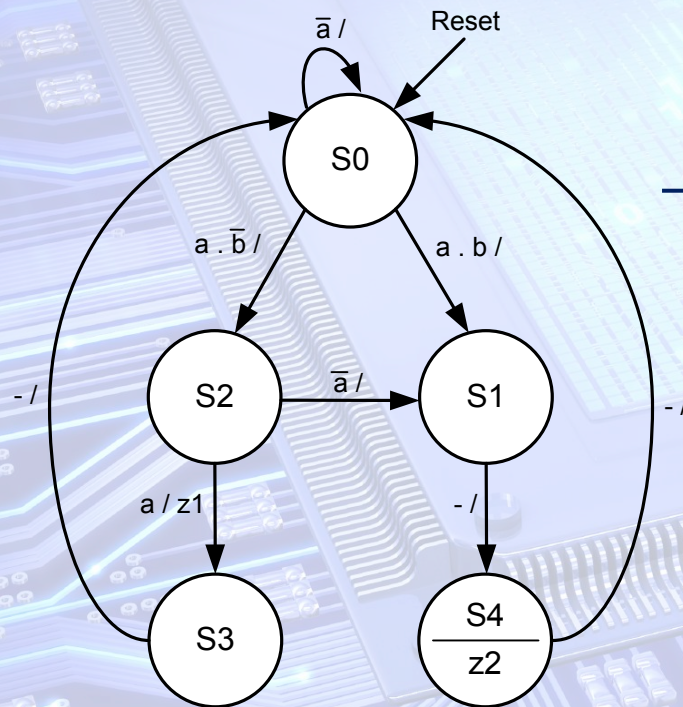
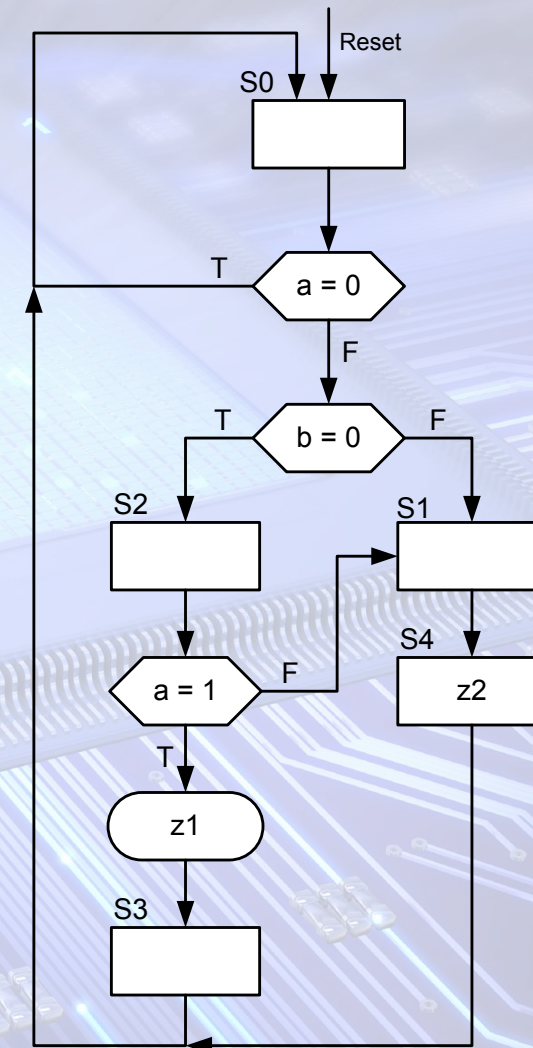


Diagrama ASM



7. Descripción de FSM. Formatos de representación. Descripción VHDL

❖ Proceso para describir una FSM en VHDL.

1. Obtener el Diagrama de Estados.

- A partir de la descripción funcional se obtiene el diagrama de estados.
- Antes de su síntesis, hay que decidir si se va a realizar como un autómata Mealy o Moore.
 - ✓ Dependiendo de la especificación funcional, se puede hacer por cualquiera de los dos tipos de autómatas, o solamente por Mealy.
 - ✓ En algunos diseños conviene mezclar ambos tipos:
 - VHDL permite describir FSM con salidas, unas de tipo Mealy y otras Moore.
- Síntesis del diagrama de estados.
 - ✓ Cada estado almacena información sobre la evolución del sistema.
 - ✓ Es muy importante comprender el funcionamiento del sistema para tener claro la información que hay que ir recordando en cada estado, de forma que no se creen estados redundantes (equivalentes).

7. Descripción de FSM. Formatos de representación. Descripción VHDL

2. Seleccionar el tipo de asignación de estados.

❖ Las herramientas de síntesis permiten dos opciones:

- Asignación automática de estados:

- ✓ La realiza la herramienta de síntesis sin la intervención del diseñador.

- ✓ Dos opciones:

- Sin atributos de usuario.

- La herramienta de síntesis realiza la asignación automáticamente seleccionando la más óptima.

- Con atributos de usuario.

- Se puede forzar un código binario: binario natural, Gray, Johnson, one hot, etc.

- Asignación específica de estados:

- ✓ El diseñador realiza la asignación manualmente.

- ✓ Dos opciones:

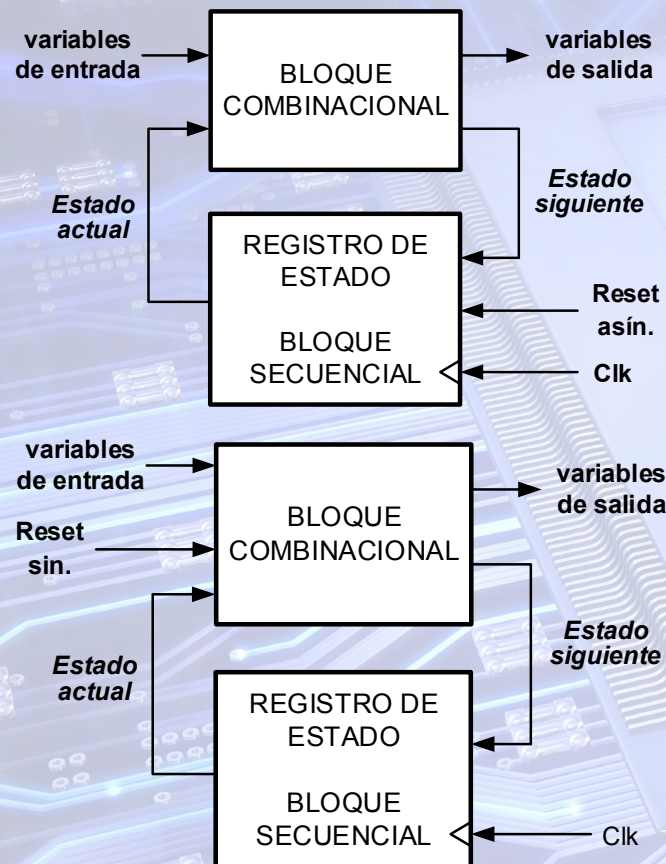
- Se debe determinar el número de variables de estado, según el código usado, y se realiza la asignación mediante constantes.

- Se usan atributos de usuario para forzar la asignación.

7. Descripción de FSM. Formatos de representación. Descripción VHDL

3. Se describe en VHDL la FSM.

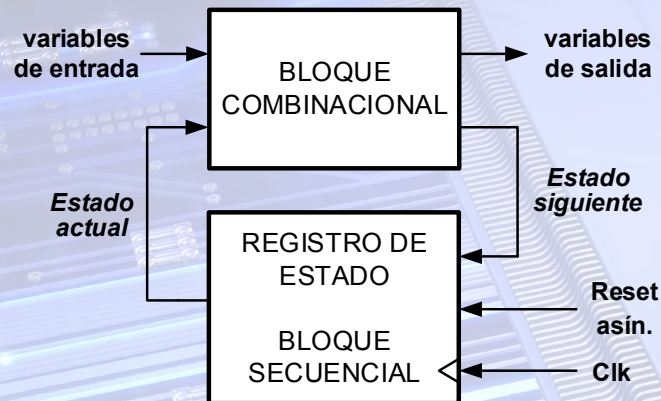
Diagramas de bloques de una FSM



- Se describen los bloques de los que consta la FSM
- Según los diagramas de bloques se usan dos process:
 - ✓ **Un process secuencial.**
 - Describe el registro de estado (estado actual).
 - ✓ **Un process combinacional.**
 - Describe el estado siguiente y el valor de las variables de salida.
 - ✓ Ambos process cambian según el **tipo de Reset.**
 - Asíncrono.
 - Síncrono.
- **Otra alternativa es describir la evolución de estados en el process secuencial.**
 - ✓ Se usará un **process combinacional** para describir el valor de las variables de salida.
 - ✓ Se usará en las **FSMDs** del tema 8

7. Descripción de FSM. Formatos de representación. Descripción VHDL

Diagrama de bloques de una FSM con Reset asíncrono



❖ Descripción VHDL de FSMs con Reset Asíncrono

- Se declaran dos señales que denominaremos **estado_actual** y **estado_siguiente**
 - ✓ El tipo de datos de ambas dependerá de si la asignación de estados es automática o especificada por el diseñador.
- Se describe el registro de estado mediante un process usando la señal **estado_actual**
 - ✓ El estado actual es:
 - El inicial si está activa la señal de restauración (Reset).
 - El estado siguiente si está desactiva Reset y se produce un flanco de reloj al que sea activa la FSM

```

Process (Clk, Reset)           -- Si el reset es asíncrono
Begin
  If Reset = '0' then           -- activo a nivel lógico bajo
    Estado_actual <= Inicio;
  Elsif Clk'event and Clk = '1' then
    Estado_actual <= Estado_siguiente;
  End if;
End process;
  
```


7. Descripción de FSM. Formatos de representación. Descripción VHDL

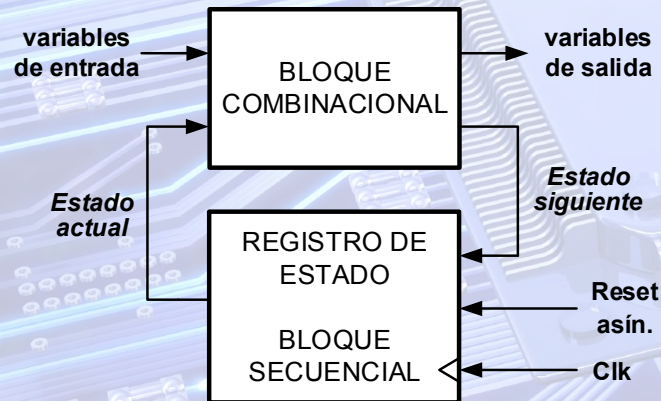
❖ Descripción VHDL de FSMs con Reset Asíncrono

- Se describe el bloque combinacional que genera el estado siguiente y el valor de las variables de salida mediante otro process.
- ✓ La descripción de los valores de las **variables de salida** dependerá de si el autómata es **Mealy** o **Moore**.

■ Aclaración:

No se puede usar un solo process, ya que el registro de estado es un circuito secuencial, y el otro bloque es combinacional.

Diagrama de bloques de una FSM con Reset asíncrono



```
Process (Estado_actual, Entradas_sincronas)
```

```
Begin
```

```
Salidas <= Valor_desactivo;
```

```
case Estado_actual is
```

```
  when Inicio =>                                -- estado inicial
```

```
    Salida1 <= '1';                                -- Moore
```

```
  If Condición then
```

```
    Estado_siguiete <= Estado2;
```

```
    Salida2 <= '1';                                -- Mealy
```

```
  Else Estado_siguiete <= EstadoN;
```

```
End if;
```

```
.....
```

```
  when others =>                                -- cubre los estados ilegales
```

```
    Estado_siguiete <= Inicio;
```

```
End case;
```

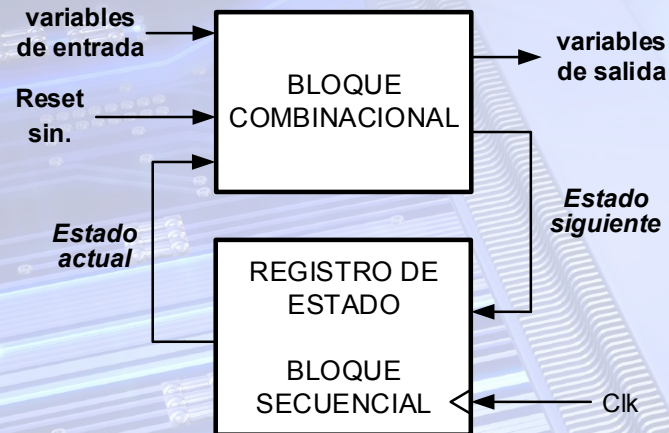
```
End Process;
```


7. Descripción de FSM. Formatos de representación. Descripción VHDL

❖ Descripción VHDL de FSMs con Reset Síncrono

- Se declaran dos señales que denominaremos **estado_actual** y **estado_siguiente**
 - ✓ El tipo de datos de ambas dependerá de si la asignación de estados es automática o especificada por el diseñador.
- Se describe el registro de estado mediante un process usando la señal **estado_actual**
 - ✓ El estado actual es siempre el estado siguiente al producirse el flanco de reloj al que sea activa la FSM.

Diagrama de bloques de una FSM con Reset síncrono

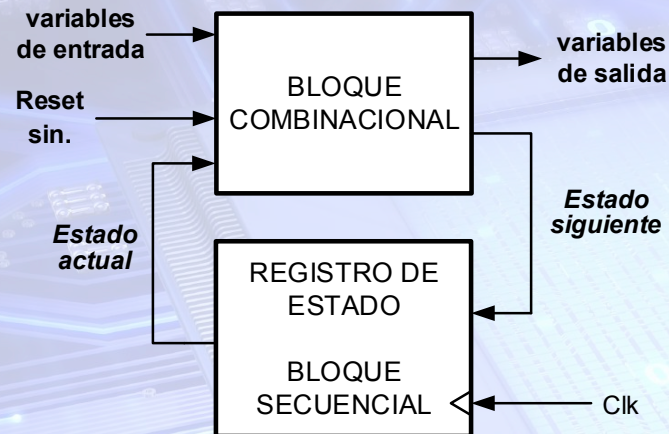


```
Process (Clk)      -- No se incluye reset al ser síncrono
Begin
    If Clk'event and Clk = '1' then
        Estado_actual <= Estado_siguiente;
    End if;
End process;
```


7. Descripción de FSM. Formatos de representación. Descripción VHDL

❖ Descripción VHDL de FSMs con Reset Síncrono

Diagrama de bloques de una FSM
con Reset síncrono



- Se describe el bloque combinacional que genera el estado siguiente y el valor de las variables de salida mediante otro process.
- ✓ La descripción de los valores de las **variables de salida** dependerá de si el autómata es **Mealy** o **Moore**.
- ✓ Si **reset** está activa el estado siguiente será el inicial.

```

Process (Estado_actual, Entradas_sincronas)  -- Se incluye Reset
Begin
  Salidas <= Valor_desactivo;
  If Reset = '1' then  -- Activo a nivel lógico alto
    Estado_siguiente <= Inicio;
  else
    case Estado_actual is
      when Inicio =>  -- estado inicial
        Salida1 <= '1';  -- Moore
        If Condición then
          Estado_siguiente <= Estado2;
          Salida2 <= '1';  -- Mealy
        Else Estado_siguiente <= EstadoN;
        End if;
      .....
      when others =>  -- cubre los estados ilegales
        Estado_siguiente <= Inicio;
    End case;
  End if;
End Process;
  
```


7. Descripción de FSM. Formatos de representación. Descripción VHDL

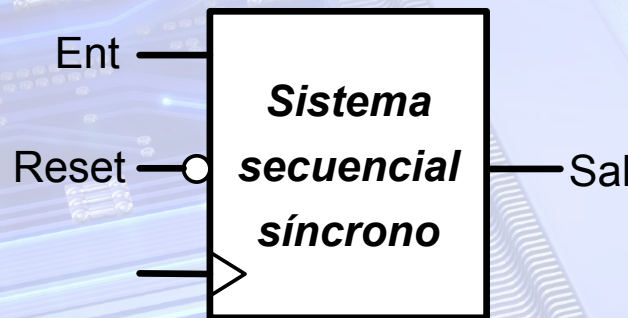
❖ Estados ilegales.

- Si el número de estados es menor que 2^r , se podrán obtener con las r variables de estado más estados de los que tiene la FSM.
- Los estados no incluidos en la secuencia válida de estados del sistema se denominan **ilegales**.
- En la descripción VHDL de una FSM se debe definir el estado siguiente y el valor de las salidas para los estados ilegales.
- Se usa **others** en la sentencia case del process que describe el estado siguiente y el valor de las salidas.
 - 💡 Se desactivan las señales de salida.
 - 💡 El estado siguiente es el estado inicial de la FSM.

9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

❖ Ejemplo de descripción VHDL de FSM

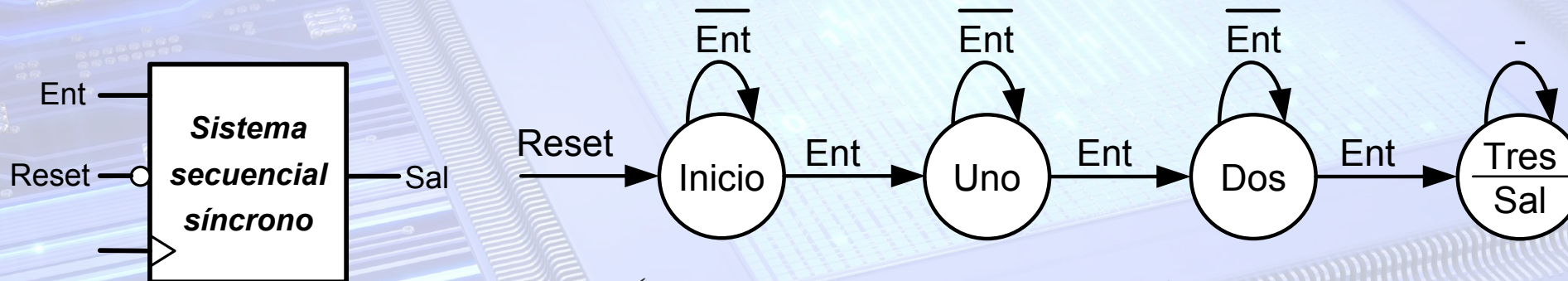
- Describir en VHDL el circuito secuencial síncrono de la figura.
 - ✓ Por la entrada **Ent** se reciben **bits en serie** sincronizados con el flanco de subida de la señal de reloj, **Clk**
 - ✓ **Reset** es una entrada de restauración asíncrona activa a nivel bajo
 - ✓ La salida **Sal** se pondrá a **1** después de recibirse **3 bits con valor 1**, teniendo en cuenta que éstos no tienen por qué ser seguidos.
 - ✓ Una vez activada la salida, ésta **permanecerá activa** hasta que se **restaure** con la señal asíncrona **Reset**
- Como **Sal** se activa después de recibirse el **3º bit con valor 1**, el diseño de la FSM se puede realizar, tanto como autómata Mealy como Moore.
- Se realiza como autómata Moore



9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

❖ Descripción detector 3 unos como autómatas Moore con Reset asíncrono

- Se sintetiza el diagrama de estados.



- ✓ Si el diagrama de estados es mínimo, es decir, no contiene estados equivalentes, se realiza su descripción en VHDL.
- ✓ De lo contrario, habría que aplicar algún método de minimización de estados, como el de la tabla de implicaciones.
- La descripción depende de si la asignación de estados es automática o especificada por el diseñador.

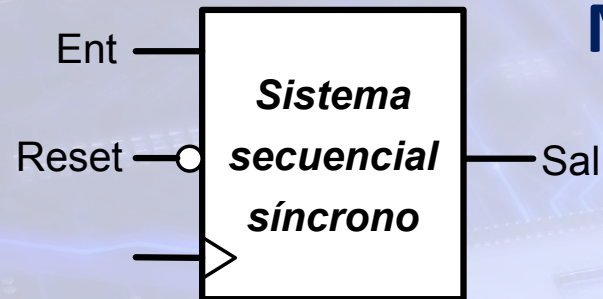
9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

- ❖ Descripción detector 3 unos como autómeta Moore con Reset asíncrono, y asignación de estados automática
 - Descripción de la entidad



```
Library ieee;  
Use ieee.std_logic_1164.all;  
Entity Detector is port (  
    Clk: in std_logic ;  
    Reset: in std_logic ;  
    Ent: in std_logic ;  
    Sal: out std_logic );  
end;
```


9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).



❖ Descripción detector 3 unos como autómata Moore con Reset asíncrono, y asignación de estados automática

▪ Descripción de los estados para la herramienta de síntesis XST de Xilinx (2)

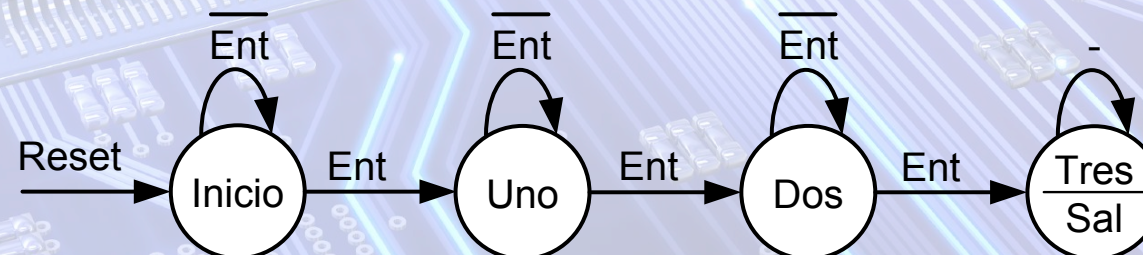
- Se declara los estados del autómata como un tipo de datos enumerado.
- Se declara dos señales para definir el estado actual y el estado siguiente del tipo de datos, estados.
- XST detecta al analizar la arquitectura que se describe una FSM y hace la asignación de estados más óptima, que generalmente es con el código one hot.

architecture Funcional of Detector is

Type Estados is (Inicio, Uno, dos, Tres);

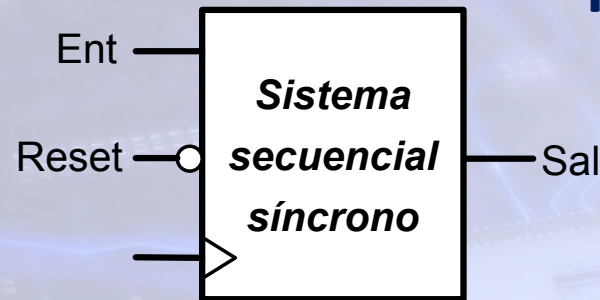
Signal Estado_actual, Estado_siguiente : Estados;

begin



9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

- ❖ Descripción detector 3 unos como autómata Moore con Reset asíncrono, y asignación de estados automática



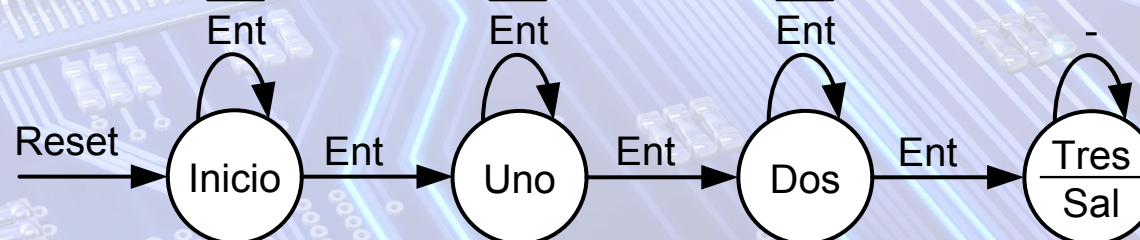
Descripción de la arquitectura (3)

- Este process describe el registro de estado.
- Reset se incluye en la lista de sensibilidad por ser asíncrono.
- Si Reset está desactivo y se produce un flanco de subida, Estado_siguiente se almacena en Estado_actual.

```

Begin
Process (Clk, Reset)
Begin
  -- Si el reset es asíncrono
  If Reset = '0' then
    -- activo a nivel lógico bajo
    Estado_actual <= Inicio;
  Elsif Clk'event and Clk = '1' then
    Estado_actual <= Estado_siguiente;
  End if;
End process;

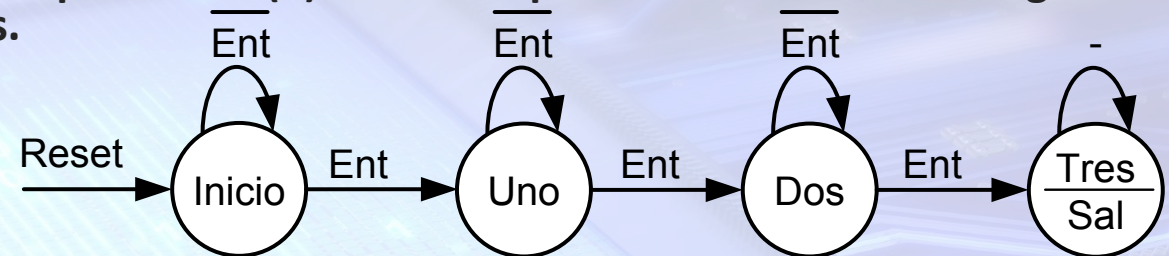
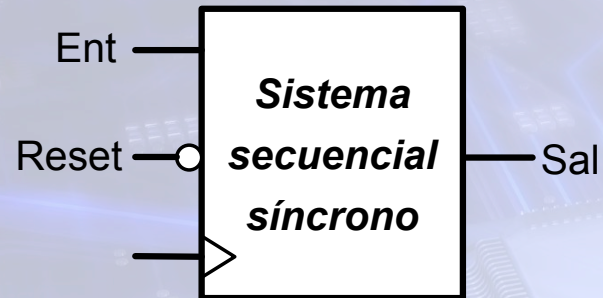
```



9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

❖ Descripción detector 3 unos como autómatas Moore con Reset asíncrono, y asignación de estados automática.

▪ Descripción de la arquitectura (4). Process que describe el estado siguiente y el valor de las salidas.



```
process (Estado_actual, Ent)
begin
  Sal <= '0';
  case Estado_actual is
    when Inicio =>
      If Ent = '1' then
        Estado_siguiete <= Uno;
      Else
        Estado_siguiete <= Inicio;
      end if;
    when Uno =>
      If Ent = '1' then
        Estado_siguiete <= Dos;
      Else
        Estado_siguiete <= Uno;
      end if;
  end case;
end process;
```

```
when Dos =>
  If Ent = '1' then
    Estado_siguiete <= Tres;
  Else
    Estado_siguiete <= Dos;
  end if;
when Tres =>
  Sal <= '1';
  Estado_siguiete <= Tres;
when others =>
  Estado_siguiete <= Inicio;
end case;
end process;
end Funcional;
```


9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

❖ Descripción detector 3 unos como autómata Moore con Reset asíncrono, y asignación de estados automática. Fichero fuente para XST (1)

```
Library ieee;  
Use ieee.std_logic_1164.all;
```

```
Entity Detector is port (  
  Clk: in std_logic ;  
  Reset: in std_logic ;  
  Ent: in std_logic ;  
  Sal: out std_logic );  
end;  
architecture Funcional of Detector is  
  Type Estados is (Inicio, Uno, dos, Tres);
```

```
  Signal Estado_actual : Estados;  
  Signal Estado_siguiente : Estados;
```

```
Begin  
  Process (Clk, Reset)      -- Si el reset es asíncrono  
  Begin  
    If Reset = '0 then  
      Estado_actual <= Inicio;  
    Elsif Clk'event and Clk = '1' then  
      Estado_actual <= Estado_siguiente;  
    End if;  
  End process;
```


9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

- ❖ Descripción detector 3 unos como autómatas Moore con Reset asíncrono, y asignación de estados automática. Fichero fuente para XST (2)

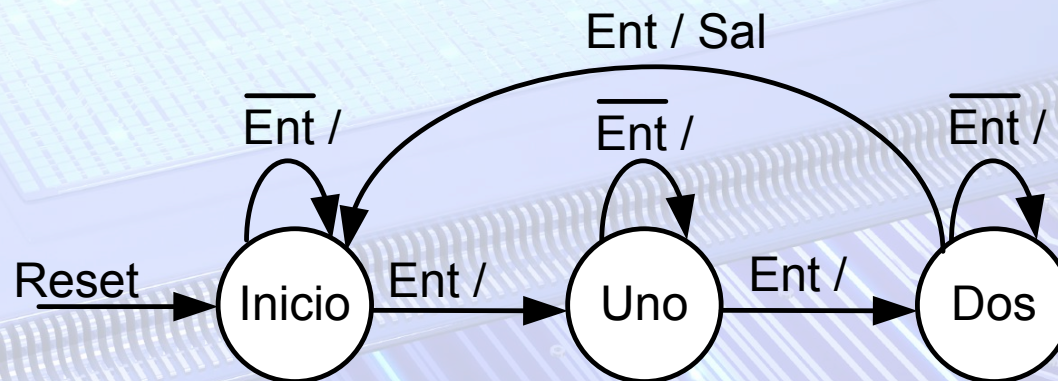
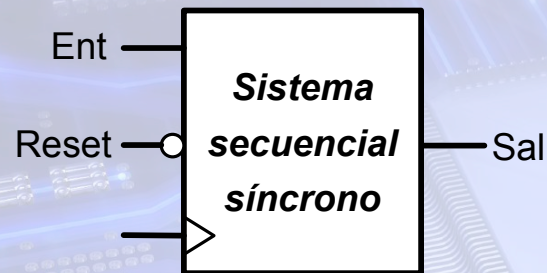
```
process (Estado_actual, Ent)
begin
  Sal <= '0';
  case Estado_actual is
    when Inicio =>
      If Ent = '1' then
        Estado_siguiente <= Uno;
      Else
        Estado_siguiente <= Inicio;
      end if;
    when Uno =>
      If Ent = '1' then
        Estado_siguiente <= Dos;
      Else
        Estado_siguiente <= Uno;
      end if;
  end case;
```

```
when Dos =>
  If Ent = '1' then
    Estado_siguiente <= Tres;
  Else
    Estado_siguiente <= Dos;
  end if;
when Tres =>
  Sal <= '1';
  Estado_siguiente <= Tres;
when others =>
  Estado_siguiente <= Inicio;
end case;
end process;
end Funcional;
```


9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

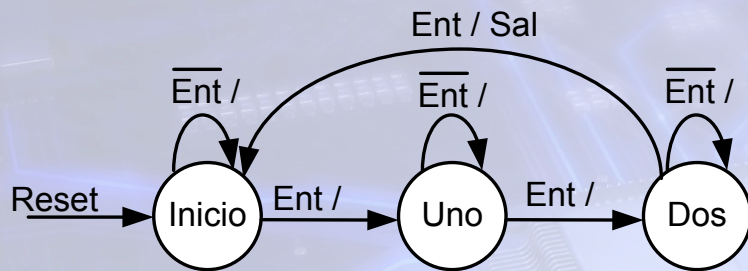
❖ Descripción detector 3 unos como autómatas Mealy con Reset síncrono, y asignación específica de estados.

- Cambio en las especificaciones:
 - ✓ Se debe detectar secuencias consecutivas de 3 unos en cualquier orden sin solape.
- Se sintetiza el diagrama de estados.



- La entidad es la misma que la del ejemplo anterior.
- Sólo se indicará la arquitectura.

9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).



❖ Descripción detector 3 unos como autómatas Mealy con Reset síncrono, y asignación específica de estados.

■ Descripción de la arquitectura (1)

- Se declara los estados como constantes, con lo que se realiza la asignación de estados.
- Se declara dos señales para definir el estado actual y el estado siguiente del mismo tipo que las constantes.
- Para que la herramienta de síntesis XST respete la descripción de los estados mediante constantes se debe usar este atributo de usuario.

architecture Funcional of Detector is

Constant Inicio : std_logic_vector(1 downto 0) := "00";

▶ **Constant** Uno : std_logic_vector(1 downto 0) := "01";

Constant Dos : std_logic_vector(1 downto 0) := "11";

-- Realiza una asignación en código gray incompleto

▶ **Signal** Estado_actual, Estado_siguiente : std_logic_vector(1 downto 0);

-- Se debe poner este atributo; de lo contrario no respeta

-- la codificación y hace una codificación one hot

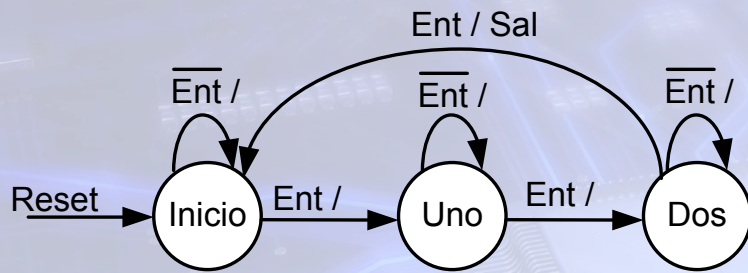
attribute FSM_ENCODING : string;

▶ **attribute** FSM_ENCODING of Estado_actual : **signal** is "USER";

9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

❖ Descripción detector 3 unos como autómata Mealy con Reset síncrono, y asignación específica de estados.

▪ Descripción de la arquitectura (2)



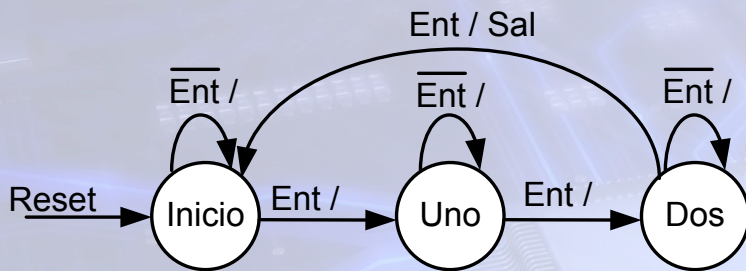
- Este process describe el registro de estado.
- La condición de restauración mediante Reset se describe en el process combinacional. En éste, si Reset está activa, se asignará Inicio a la señal Estado_siguiente.
- Si se produce un flanco de subida:
 - El Estado_siguiente se almacena en Estado_actual.

```
Begin
  Process ( Clk )
  Begin
    If Clk'event and Clk = '1' then
      Estado_actual <= Estado_siguiente;
    End if;
  End process;
```


9. Ejemplos de descripción en VHDL de Máquinas de Estados Finitas (FSM).

❖ Descripción detector 3 unos como autómatas Mealy con Reset síncrono, y asignación específica de estados.

■ Descripción de la arquitectura (3)



```

process (Estado_actual, Ent, Reset)
begin

```

```

  Sal <= '0';

```

```

  If Reset = '0' then

```

```

    Estado_siguiete <= Inicio;

```

```

  Else

```

```

    case Estado_actual is

```

```

      when Inicio =>

```

```

        If Ent = '1' then

```

```

            Estado_siguiete <= Uno;

```

```

        Else

```

```

            Estado_siguiete <= Inicio;

```

```

        end if;

```

- Si reset está activa el estado_siguiete (E.S.) será Inicio.

- De lo contrario, el case determina E.S. según el valor de Estado_actual y Ent.

- Al ser Mealy, la salida depende de la entrada.

- Por ello, se comprueba Ent con If, y se activa si Ent está a 1 lógico ('1').

```

when Uno =>

```

```

    If Ent = '1' then

```

```

        Estado_siguiete <= Dos;

```

```

    Else

```

```

        Estado_siguiete <= Uno;

```

```

    end if;

```

```

when Dos =>

```

```

    If Ent = '1' then

```

```

        Estado_siguiete <= Inicio;

```

```

        Sal <= '1';

```

```

    Else

```

```

        Estado_siguiete <= Dos;

```

```

    end if;

```

```

when others =>

```

```

    Estado_siguiete <= Inicio;

```

```

end case;

```

```

End if;

```

```

end process;

```

```

end Funcional;

```


10. Asignación de estados mediante atributos de usuario

• Atributos de usuario

- Para usar un atributo de usuario, primero debe declararse y después especificarse.

▪ Sintaxis de declaración de un atributo

Attribute nombre_atributo : tipo_dato;

▪ Sintaxis de especificación de un atributo

Attribute nombre_atributo of nombre_elemento : clase is valor;

- ✓ **Tipo_dato** es el tipo de dato del valor del atributo: integer, std_logic, boolean, etc.
- ✓ **Clase del elemento**, puede ser type, signal, function, etc.

❖ Herramienta de síntesis XST

- Asignación de estados mediante el atributo **fsm_encoding**.
 - ✓ Especifica el tipo de codificación de estados a XST.
 - ✓ Opciones: Auto, one-hot, compact, sequential, gray, johnson, speed1 y user
- También se puede configurar XST con el código correspondiente.
- **Ejemplo:** asignación de estados en el código Gray

```
Type Estados is (Inicio, Uno, dos, Tres, Cuatro);  
Signal Estado_actual, Estado_siguiente : Estados;  
-- El atributo fsm_encoding indica a XST el tipo  
de codificación  
attribute fsm_encoding : string;  
attribute fsm_encoding of Estado_actual: signal  
is "gray";
```


10. Asignación de estados mediante atributos de usuario

❖ Herramienta de síntesis XST

- Asignación específica de estados mediante el atributo **enum_encoding**

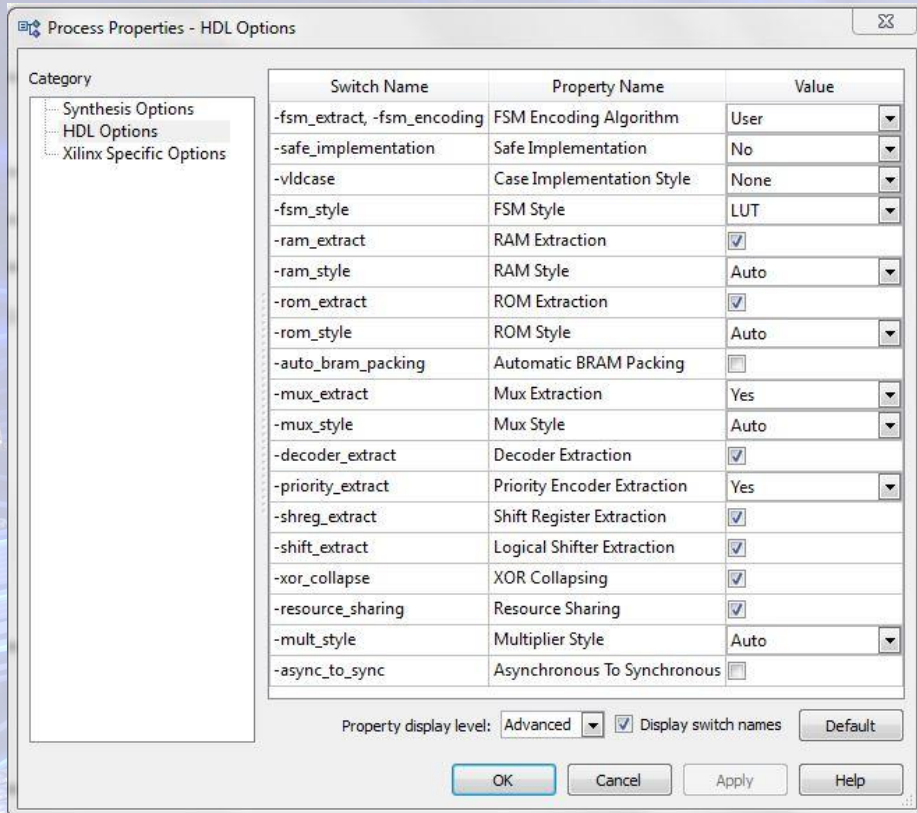
- Se declara los estados del autómata como un tipo de datos enumerado.
- La asignación de estados se realiza mediante el atributo **enum_encoding**.
- Se declara dos señales para definir el estado actual y el estado siguiente.
- Con este atributo de usuario se fuerza a la herramienta de síntesis XST a que respete la asignación de estados.
- De lo contrario, XST trata de encontrar la más idónea, que suele ser en el código one hot.

```
Type Estados is (Inicio, Uno, dos, Tres, Cuatro);  
Attribute enum_encoding : string;  
Attribute enum_encoding of Estados : type is "000 001 011 111 110";  
-- Se debe poner este atributo después de la declaración de tipo  
Signal Estado_actual, Estado_siguiente : Estados;  
-- Se debe poner este atributo; de lo contrario no respeta la codificación  
attribute fsm_encoding : string;  
attribute fsm_encoding of Estado_actual : signal is "user";  
-- Le indica a XST que no optimice la asignación de estados y  
-- que respete la asignación realizada por el usuario  
-- Opciones del atributo fsm_encoding: auto, one-hot, compact,  
sequential, gray, johnson y user
```


10. Asignación de estados mediante atributos de usuario

❖ Herramienta de síntesis XST

- Configurando adecuadamente XST, no es necesario usar el atributo **fsm_encoding**.
 - ✓ Se puede configurar con los mismos valores del atributo **fsm_encoding**.
 - ✓ Configurando XST con el valor **user** solo se necesita el atributo **enum_encoding**.



Type Estados is (Inicio, Uno, dos, Tres, Cuatro);

Attribute enum_encoding : string;

Attribute enum_encoding of Estados :

type is "000 001 011 111 110";

-- Se debe poner este atributo después de la

-- declaración de tipo: Estados

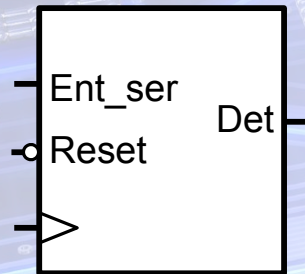
Signal Estado actual, Estado siguiente : Estados;

11. Problemas

❖ Diseño 1

- Describir en VHDL el circuito secuencial síncrono de la figura. Además de la señal de reloj tiene dos entradas:
 - ✓ **RESET**. Entrada de restauración. Es asíncrona y activa a nivel lógico bajo.
 - ✓ **Ent_ser**. Entrada por donde se reciben en serie bits sincronizados con el flanco de subida de CLK
- Genera un pulso en la salida **Det** después de recibirse la secuencia **10110**, teniendo en cuenta que ésta puede ir precedida por cualquier patrón de bits, y que primero se recibe el bit de la izquierda. El circuito debe detectar secuencias consecutivas.

Detec. secuencia

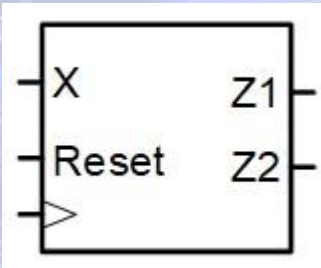


❖ Diseño 2

- Repítase el diseño 1, considerando las siguientes condiciones:
 - ✓ Reset es síncrona.
 - ✓ La salida se activará al recibirse el último bit.
 - ✓ Para la detección se considerará el solape entre dos secuencias consecutivas. Se trata de un detector con solape.

11. Problemas

❖ Diseño 3

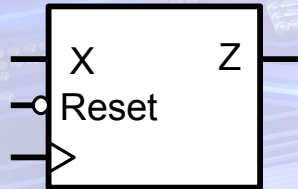


- Describir en VHDL la FSM del circuito de la figura. La descripción se realizará para la herramienta XST, y realizando una asignación automática de estados. **RESET** es la entrada de restauración asíncrona, y por **X** se reciben en serie secuencias de bits sincronizados con el flanco de subida de la señal de reloj, **CLK**. La salida **Z1** se pondrá a 1 al recibir el cuarto bit de la secuencia **0101**, y la salida **Z2** al recibir el cuarto bit de la secuencia **1110**. Cada secuencia válida puede ir precedida de cualquier patrón de bits. Realícese el diseño teniendo en cuenta que deben detectarse secuencias consecutivas.

11. Problemas

❖ Diseño 4

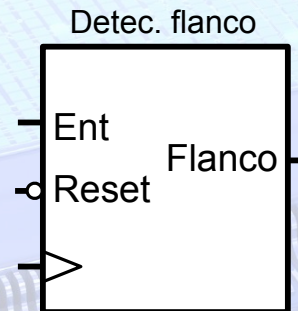
- Diseñar el circuito secuencial síncrono de la figura. **RESET** es la entrada de restauración síncrona activa a nivel lógico bajo, y **X** una entrada por donde se reciben en serie caracteres **BCD** sincronizados con el flanco de subida de la señal de reloj. La salida **Z** se activará al recibir el último bit de los caracteres con **valor 5 o 7**. Realícese el diseño teniendo en cuenta que los caracteres BCD se reciben consecutivamente y que primero se recibe el bit menos significativo de cada carácter.



11. Problemas

❖ Diseño 5

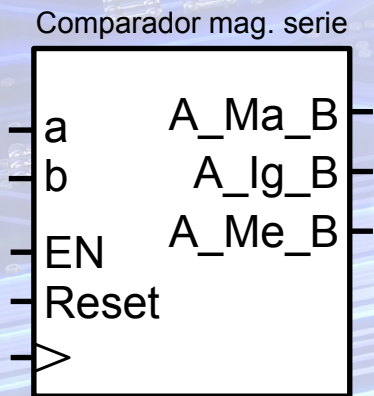
- Describir en VHDL, como una FSM, el circuito de la figura. Éste es un detector de flanco de subida. En la salida **Flanco** se generará un pulso a nivel alto durante un periodo de reloj cada vez que se produzca un flanco de subida en la entrada **Ent**. **Reset** es la entrada de restauración síncrona.



11. Problemas

❖ Diseño 6

- Describir en VHDL como una FSM el comparador de magnitud serie de números binarios de tres bits de la figura. La descripción se realizará para la herramienta XST y teniendo en cuenta que se realiza una asignación automática de estados. El sistema tiene cuatro entradas **Reset, a, b y EN**, y tres salidas, **A_Ma_b, A_Ig_B y A_Me_B**. Reset es la entrada de restauración síncrona, que es activa a nivel lógico alto. En a y b se reciben síncronamente con el flanco de subida de la señal de reloj los bits de cada número. Al activarse EN indica que se está recibiendo el primer bit válido de cada número. EN es activa a nivel alto. Realícese el diseño teniendo en cuenta:
 - ✓ Primero se recibe el bit más significativo.
 - ✓ Una vez recibido el primer bit de cada número, si se desactiva EN se debe continuar con la comparación de los números hasta que se reciban los tres bits.
 - ✓ Las salidas estarán a 0 lógico hasta el momento en el que se recibe el tercer bit de ambos números, en el cual se activará la salida correspondiente. Una vez recibidos los números el sistema permanecerá bloqueado con la salida correspondiente activa, hasta que se restaure mediante una señal de restauración síncrona (RESET).



11. Problemas

❖ Diseño 7

Árbitro de bus



- Describir en VHDL como una FSM el árbitro de bus de la figura. **Req0, Req1 y Req2** son las peticiones de bus y **Ack0, Ack1 y Ack2** las correspondientes concesiones de bus. Si se activa más de una petición de bus al mismo tiempo, se concederá el bus a la petición más prioritaria, siendo Req0 la más prioritaria y Req2 la menos prioritaria. Una vez concedido el bus a un dispositivo, se mantiene la señal de concesión activa hasta que éste desactive su petición, aunque ésta sea la menos prioritaria y esté activa otra más prioritaria. Se realizará el diseño de forma que se optimice el diseño lo máximo posible, reduciendo el número total de señales necesarias (variables de estado + señales de salida). El alumno tendrá en cuenta que este requisito puede forzar una asignación específica de estados. De ser así, se realizará mediante constantes. La señal de restauración, RESET, es síncrona y activa a nivel lógico bajo. La descripción se realizará para la herramienta XST.

11. Problemas

❖ Diseño 7

Árbitro de bus

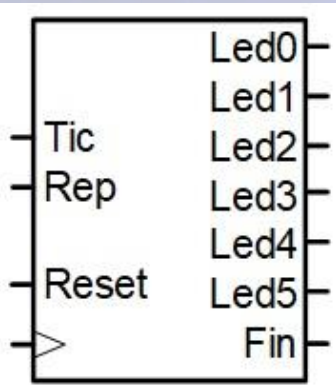


- Describir en VHDL como una FSM el árbitro de bus de la figura. **Req0, Req1 y Req2** son las peticiones de bus y **Ack0, Ack1 y Ack2** las correspondientes concesiones de bus. Si se activa más de una petición de bus al mismo tiempo, se concederá el bus a la petición más prioritaria, siendo Req0 la más prioritaria y Req2 la menos prioritaria. Una vez concedido el bus a un dispositivo, se mantiene la señal de concesión activa hasta que éste desactive su petición, aunque ésta sea la menos prioritaria y esté activa otra más prioritaria. Se realizará el diseño de forma que se optimice el diseño lo máximo posible, reduciendo el número total de señales necesarias (variables de estado + señales de salida). El alumno tendrá en cuenta que este requisito puede forzar una asignación específica de estados. De ser así, se realizará mediante constantes. La señal de restauración, RESET, es síncrona y activa a nivel lógico bajo. La descripción se realizará para la herramienta XST.

11. Problemas

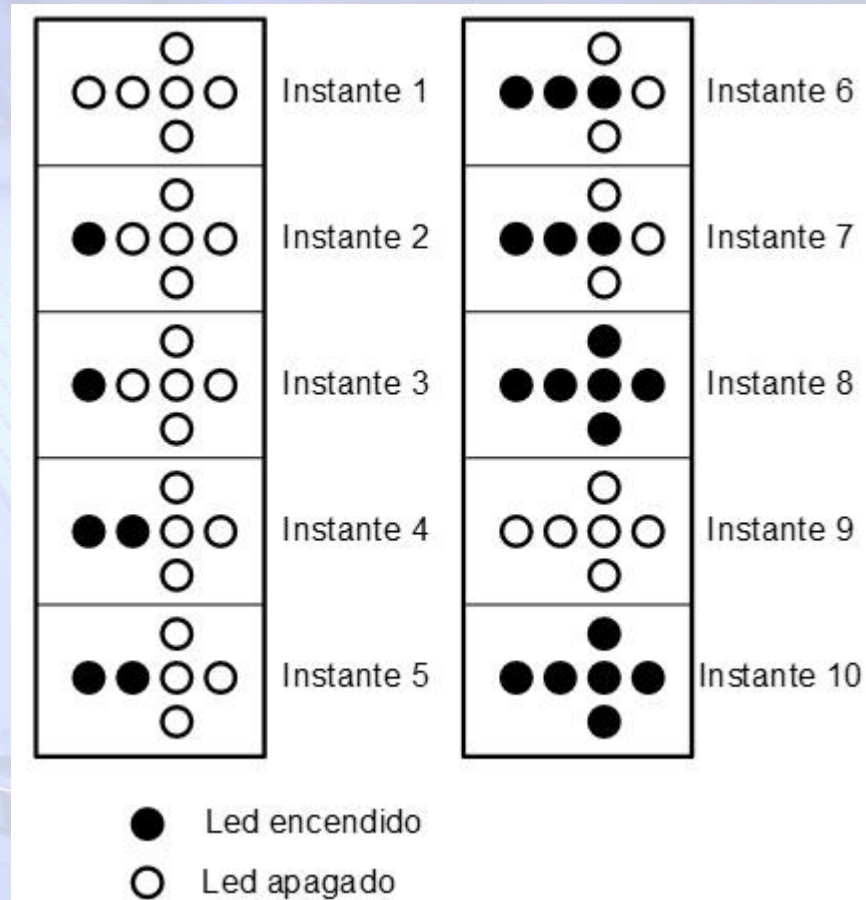
❖ Diseño 8

- Diseñar mediante una FSM el circuito de control de un cartel de efectos luminosos. En el cartel se debe visualizar una flecha mediante LEDs, los cuales se han de encender y apagar en la secuencia que se indica en la figura.
- ✓ **Reset** es la entrada de inicialización asíncrona, de forma que al activarse el sistema se inicializa al instante 1.
- ✓ **Tic** es una señal de tipo pulso, de forma que cuando se pone a 1, se debe pasar de un instante a otro.
- ✓ Si **Rep** está a 1 se eliminan los instantes 3, 5 y 7.
- ✓ **Fin** se activará cuando se esté en el instante 10.
- ✓ El Led5 es el de la izquierda, y Led2, Led1 y Led0, los de la punta de la flecha.



11. Problemas

❖ Diseño 8



11. Problemas

❖ Diseño 9

- Describir en VHDL el diagrama de estados de la FSM de la figura. Se hará la descripción de forma que sea lo más reducida posible. Téngase en cuenta que el Reset es asíncrono, y que la FSM es activa por flanco de bajada. Se realizará una asignación automática de estados.

